blunderDB

Release 0.32.0

Kevin UNGER <blunderdb@proton.me>

Aug 09, 2026

CONTENTS

blunderDB is a software for creating backgammon position databases. Its main strength is to provide a single place where a player can aggregate the positions he or she has encountered (online, in tournaments) and be able to review these positions by filtering them with various filters that can be arbitrarily combined. blunderDB can also be used to create catalogs of reference positions.

The present documentation is structured as follows:

- the **download and installation** section explains how to obtain and run blunderDB.
- the **manual** describes the general functioning of blunderDB and how it should be used.
- the **user guide** is a practical introduction for quickly using blunderDB.
- the list of **commands** as well as the list of **keyboard shortcuts** enables efficient use of blunderDB.
- the **command line interface (CLI)** section describes the available commands for bulk import, automation, and scripting.
- The **FAQ** provides answers to the most frequently asked questions.

CHAPTER

ONE

VERSION HISTORY

Version	Date	Cause and/or nature of changes
0.1.0	December 31, 2024	Beta version
0.2.0	January 6, 2025	Various bug fixes. Addition of match/TP/GV tables. Added search filters (moves, cube decisions, date). Added metadata for positions. Import/export functionality between blunderDB instances. Added metadata functionality for databases. Introduction of version numbers (database and application).
0.3.0	January 27, 2025	Various bug fixes. Automatically saves the window size. Imports any comments from XG.
0.4.0	February 3, 2025	Various bug fixes. Adding an icon for blunderDB. Filter corrections. Adding macOS support.
0.5.0	February 4, 2025	Addition of new filters (mirror, non-contact, jan blot, outfield blot).
0.6.0	February 13, 2025	Add filter library. Displaying the database version in the metadata.
0.7.0	February 16, 2025	Support Japanese and German XG exports.
0.8.0	May 3, 2025	New button to hide pipcount. Button to load random position.
0.9.0	November 02, 2025	Fix filter library bug. Database import/export. Display arrows for selected moves. Keyboard shortcuts for import/export.

continues on next page

Table 1 – continued from previous page

Version	Date	Cause and/or nature of changes
0.10.0	February 25, 2026	Match import from eXtreme Gammon (XG/XGP), GNUbg (SGF), Jellyfish (MAT/TXT) and BGBlitz (BGF/TXT). Match navigation: browse through moves of an imported match, with played move highlighting. Match panel: list, sort, inline editing, player swap, tournament assignment. Recursive folder import and drag & drop import. EPC (Effective Pip Count) calculator with built-in GNUbg bearoff database. Collections: custom position grouping. Tournaments: group matches by event. Save and restore session state (last search, current position). Automatic database schema migration. Multi-engine display in analysis. Player 1 error/blunder filter in searches. Database export with granular selection (matches, collections, tournaments, played moves). Match navigation button. Pipcount display in match navigation. Complete command-line interface (CLI). Automatic reopening of the last database. Improved toolbar and icons.
0.11.0	March 6, 2026	Filter to search within currently filtered positions. Add match and tournament filters. Automatic board clearing when opening the search panel.
0.12.0	March 19, 2026	Import eXtreme Gammon position files (XGP) with analysis.
0.13.0	March 28, 2026	Interface simplification: match and collection navigation is now done directly through panels. Command line integrated into the status bar. Console panel renamed to Log panel. Dedicated Bearoff panel in the bottom panel. Copy/paste position in the search panel. Drag and drop to reorder collections, positions within collections, and matches within tournaments. Tournament column in the match panel with inline editing. Automatic display of the analysis panel after a search.
0.14.0	March 30, 2026	Dedicated Anki panel for spaced repetition study (FSRS algorithm). Import comments from XG files.
0.15.0	March 31, 2026	Export position as PNG image to clipboard (board only via Ctrl+X, or board with analysis via Ctrl+X Ctrl+X).
0.16.0	April 18, 2026	Database schema v2.0.0: Zobrist-hashed position deduplication, denormalized filter columns, bitboard pattern pre-filter, WAL journaling. Batch import >=3x faster, filtered search <=100 ms on 10k+ positions. NOTE: DB files created with v0.16.0 cannot be opened by older versions; old DBs are auto-migrated in place (back up first).

continues on next page

Table 1 – continued from previous page

Version	Date	Cause and/or nature of changes
0.17.0	April 20, 2026	Storage optimization: zlib compression of analysis data (~80% reduction), compact board encoding (~90% position state size reduction). Added 5 missing indexes for search performance. Fixed cube error search returning wrong results. Fixed EDIT mode after search returns no results. Restored search panel state when switching tabs. Removed 62 debug print statements from hot search/filter paths.
0.18.0	April 20, 2026	Major codebase refactoring: split db.go (10k lines) into 19 domain-focused files, extracted 7 service modules from App.svelte (4,888→469 lines), consolidated modal/panel stores. Full migration to Svelte 5 runes. Replaced 9 table modals with a generic DataTableModal component. Added ESLint + Prettier + vitest (125 frontend tests) with CI enforcement. WCAG 2.1 AA accessibility baseline (visible focus, ARIA roles, keyboard navigation). Upgraded Database mutex to RWMutex for better read concurrency. Complete CLI documentation (CLI_USAGE.md + Sphinx FR/EN). Rewrote README. Resolved all ESLint warnings (46→0) and Vite build warnings (6→0).
0.19.0	May 7, 2026	Added Stats panel: PR (Performance Rate), Snowie Error Rate and MWC cost (Match Winning Chance cost) indicators, filter bar (player, tournament, dates, decision type, match length), Dashboard tab with level cards / rolling PR / top blunders, Progression tab with per-tournament curve and per-match scatter plot, Errors tab with cube-action breakdown and error magnitude histogram. Interactive drill-down to positions / matches / tournaments from all indicators. Instant PR / MWC toggle. CLI command list –type stats. Alignment of PR / Snowie ER / MWC indicators with eXtremeGammon and gnuBG (0.16 equity threshold for close cubes). Fixed cube_error calculation for Double/Pass decisions. Statistics model documentation (*Annex: Statistics model — XG / gnuBG / blunderDB alignment*). See *Stats Panel*.
0.20.0	May 31, 2026	Added the *Except* exclusion structure to the search panel: excludes positions that contain any of the drawn checkers, with a « must be empty » marker (double-click) and no limit on the number of checkers per point (command x). Added a « first die only » option to the dice roll filter (D1 variant, –dice CLI option). Comments panel: the input field is focused automatically on open and the edit / delete buttons are always visible. Fixed the « Search Text » filter that failed to match all comment tags.
0.21.0	June 1, 2026	Interface internationalisation: the whole of blunderDB (toolbar, panels, messages, help) can now be displayed in English, French, German, Italian, Spanish, Finnish, Japanese, Greek or Russian, as you choose. Added a configuration window, accessible through a gear-shaped button in the toolbar, to select the language. The language choice is kept across sessions. See *Configuration*.

continues on next page

Table 1 – continued from previous page

Version	Date	Cause and/or nature of changes
0.22.0	June 2, 2026	Added a headless (server) mode, advanced and optional, complementing the desktop application: a `serve` daemon exposing the engine over HTTP + JSON, a multi-user PostgreSQL backend with per-tenant isolation (and optional Row-Level Security), a `migrate` command to transfer a SQLite database to PostgreSQL, and a generic `call` dispatcher giving command-line access to every storage operation. Import of single positions from new formats (eXtreme Gammon `.xgp`, BGBlitz text, native `.db` library) with enrichment of duplicates across formats. Fixes: panels no longer raise an error when no database is open, and the Comments panel no longer loops on opening. See *Headless mode (server)*.
0.23.0	June 5, 2026	Added guided tours of the interface (general tour, search, matches, tournaments), replayable from the toolbar or with the `tour` command, and a sample database loadable with the `demo` command to explore the tool without importing your own games. Customisation of the board colours (background, border, points, checkers, dice, doubling cube) from the configuration window. Command-line autocompletion (*TAB* key). Search panel: explicit control over the decision type (Checker / Cube), with a Double / No double or Take / Pass sub-type for cube decisions, synchronised with the board; the offered cube is shown in the centre of the board for take/pass decisions. Search for a position by its identifier (`id` filter). Fixed the attribution of the cube error to player 1. See *Guided tours and sample database*.
0.24.0	June 6, 2026	Added a container image (Docker) for headless mode: a dedicated `serve` entry point and a `Dockerfile.serve` producing a static binary with no graphical interface, to deploy the blunderDB engine as a service behind a reverse proxy. See *Headless mode (server)*.
0.25.0	June 7, 2026	Server (headless) mode: two new methods to rebuild a position without storing it — `positions.fromXGID` decodes an XGID string into a position, and `positions.fromXGP` reads a single-position `.xgp` file. See *Headless mode (server)*.
0.26.0	June 9, 2026	Interface display settings in the configuration window: a scale slider to enlarge or shrink the whole interface, and a panel position choice (bottom, side or automatic) to make better use of wide screens. Added an information bar above the board showing the players and the match context (event, location, round, date, length). When a database is opened, the matches panel is shown right away and the review starts on the first position. Keyboard shortcuts are now independent of the keyboard layout (AZERTY, QWERTY, QWERTZ…). Fixed XGID decoding (Jacoby/Beaver flags and cube value). Added multiple view tabs: several independent workspaces (position list, current position, search) with the shortcuts *CTRL-T* (new view), *CTRL-W* (close), *CTRL-PageUp*/*CTRL-PageDown* (switch view) and renaming by double-clicking the tab. See *Configuration* and *View Tabs*.

continues on next page

Table 1 – continued from previous page

Version	Date	Cause and/or nature of changes
0.27.0	June 13, 2026	Anki panel: a new free-drill (*cram*) mode, available from the *Cram* button next to *Study*, which serves random positions from the deck without altering the spaced-repetition schedule — ideal for warming up before a tournament or intensively reviewing a deck without disturbing its ordering. Search: a new `blunders` command (alias `bl`) that loads your worst mistakes (equity/MWC) straight into the analysis view, with an optional number to choose how many (`bl 50`); a new player filter `pl'name'` that finds every position from a match involving a given player, on either side; and tooltips that, on hovering each search-panel filter, show the corresponding command token. Server (headless) mode: new `matches.movesByMatch` (all the moves of a match in a single call) and `positions.epc` (Effective Pip Count for both players) methods. See *Anki Panel* and *List of commands*.
0.28.0	July 15, 2026	Search panel: unified "Matches & Tournaments" filter replacing the free-text ID fields with a picker window (text-filterable checkbox lists, *All*/*None* buttons, checking a tournament checks its member matches) — faster on large databases, since the Matches tab and Tournaments view now only recompute PR/MWC badges for the matches actually displayed; `cli list --type tournaments` closes the matching parity gap. New "Individually Imported" filter (checkbox in the Other group, or the `i` command-line token, `s i`; also available in the CLI via `--individual`) to find a position that was saved or imported on its own rather than buried in a match import. Fixed a data-loss bug: deleting a match no longer deletes positions that were individually imported, added to a collection, or added to an Anki deck, even when the deleted match also contained them. Server mode (headless): six new methods for the Anki spaced-repetition scheduler — review log (`anki.reviewLog`), due-card forecast (`anki.forecast`), suspend / bury / remove a card (`anki.suspendCard`, `anki.buryCard`, `anki.removeCard`) and automatically nudging a deck's target retention rate toward its observed pass rate (`anki.optimizeParams`); new `tenant.purge` method to permanently delete a tenant's data on a multi-user PostgreSQL deployment; new `matches.list` (filter/sort/paginate) and `stats.matchBadges` (PR/MWC badges scoped to a list of matches) methods. Native Linux distribution: `.deb`/`.rpm` packages, which set the execute bit that was lost when downloading the raw binary. Import: after importing a match, the Matches tab now displays and the imported positions are immediately visible; after importing a standalone position (XGP file, BGBlitz position, position text file, pasted position, or a batch of positions), the Analysis tab now opens directly on the imported position instead of the Matches tab; fixed pasting a French or German eXtreme Gammon analysis from macOS, which came back empty (accents decomposed by the system clipboard). See *Headless mode (server)*, *List of commands* and *Download and Installation*.

continues on next page

Table 1 – continued from previous page

Version	Date	Cause and/or nature of changes
0.29.0	July 18, 2026	Export of a match to the Jellyfish/gnubg format (`.mat`) from the interface (Matches panel) as well as from the command line, with the corresponding server route — handy for replaying a match in another backgammon program. Search: a new "excluded dice" discriminator (`xD65` token, several allowed) that discards positions whose roll matches a given throw. Safe concurrent opening: a database already open for writing in another blunderDB window now opens read-only (navigation, search and analysis still possible, edits disabled, "[read-only]" shown in the title bar) instead of risking corruption. Position provenance: outside match mode, the information bar above the board indicates the match — and its tournament — the studied position comes from, with a "+N" badge when it appears in several matches. Increased robustness to the environment: automatic fallback when a font, the clipboard or the last opened database is unavailable. An explicit reload of the library (`CTRL-R`, toolbar button or `e` command) reopens the analysis panel of the displayed position. Notable fixes: removal of a freeze when exporting large databases, the export "None" button that did not actually export zero matches, multi-word comment filter, deduplication of the event and tournament in the information bar, and various display adjustments. See *List of commands*, *Matches Panel* and *Manual*.
0.30.0	July 26, 2026	Comment search: the "Search Text" filter becomes the "Comment" filter and offers three exclusive modes — *contains text* (previous behaviour, token `t""word1;word2""`), *has a comment* (token `co`) and *no comment* (token `xco`) — to find annotated positions at a glance, or on the contrary those left to comment. Positions flagged in eXtreme Gammon: the flags set while analysing an XG match are now read on import and found again with the "Flagged" filter (token `fl`), combinable with every other criterion; a flagged cube decision yields two flagged positions, the double and the take/pass. Flagging is not retroactive, so simply import the relevant `.xg` file again for its flags to join the database, leaving existing comments and analyses untouched. Notable fixes: the PR shown on a tournament badge refers to the reference player rather than the two players combined, Anki decks based on a search again honour that search's filters instead of taking in the whole database, and a search hang combining certain filters (text, date, played-move error) is fixed. See *List of commands* and *Manual*.

continues on next page

Table 1 – continued from previous page

Version	Date	Cause and/or nature of changes
0.31.0	August 4, 2026	Handing a database to other people: two independent mechanisms, both optional and both chosen at export time. The **origin watermark** records in the file what it is and where it comes from, signed by an issuer identity created by itself on first use; the mark is unforgeable, is read at the top of the Metadata panel (or with `blunderdb info`, which never writes to the file) and records nothing on the recipient's side — no log, no tracking. **Password protection** produces an encrypted `.dbx` file (AES-256-GCM, key derived with Argon2id), checked on every open, whose header stays readable in the clear; the identity is exported and imported as a single file from the preferences. Redesigned export dialog: one screen instead of four, a reminder of what actually goes out (the positions on display), the covered share of each collection flagged in red when partial, tournaments exportable only together with their matches — and an export three times faster. Settings window reorganised into three tabs (Interface, Board colours, Issuer identity), with a stable size and position. Metadata panel laid out as a single grid. The Log panel, which only echoed commands, has been removed. Notable fixes: a wrong password no longer opens a protected database, modal dialog fields no longer lose focus on click, the PR/MWC badge computation no longer crashes on a money game, and temporary files (`-wal`, `-shm`, lock) are cleaned up on close. See *Handing out a database: origin and password* and *Manual*.
0.32.0	August 9, 2026	The EPC panel becomes the **Bearoff** panel and gains full race analysis. On a pure bearoff position it displays — in addition to each player's EPC, pip count, wastage and average number of rolls, now laid out as a table with a signed Δ row — both players' winning probabilities and the money equities (cubeless, no double, double/take, double/pass, with each decision's gap to the best one) as well as the **best cube decision**. Two clearly distinguished regimes: *exact* (looked up in a two-sided database; GNUbg's TS-06-06 database built in, covering up to 6 checkers per player) and *estimated* (calibrated convolution, error margin displayed — the cube verdict is never estimated). The extended TS-06-11 database (exact verdict up to 11 checkers per player, 1.2 GB) can be downloaded from the new *Bearoff* tab of the configuration, with integrity checking and resuming of interrupted downloads; any two-sided gnubg `.bd` file can also be pointed to. **Challenge mode** for training: results are hidden on every change of the position and revealed area by area, with a click. Editing entirely on the board: checkers of both home boards, player on roll (click the bearoff/score rectangle) and cube position (click the cube). Clicks on the board are now accurate at any interface scale (a click on point 1 could land on point 2). New `blunderdb epc` command on the command line and `--bearoff-ts` server-mode option. A new manual section, "Methodology and assumptions of the Bearoff panel", exhaustively states the assumptions behind every displayed value. See *Bearoff Panel* and *Methodology and assumptions of the Bearoff panel*.

CHAPTER

TWO

TABLE OF CONTENTS

2.1 Download and Installation

blunderDB is a single executable that requires no installation.

The latest version of blunderDB is available under the MIT license:

- for Windows: https://github.com/kevung/blunderDB/releases/latest/download/blunderDB-windows-0.32.0.exe
- for Linux: https://github.com/kevung/blunderDB/releases/latest/download/blunderDB-linux-0.32.0
- for Mac: https://github.com/kevung/blunderDB/releases/latest/download/blunderDB-macos-0.32.0.zip

Note

blunderDB uses Webview2 for rendering the graphical interface. It is likely that Webview2 is already present natively on your operating system. If not, the first execution of blunderDB will offer to download and install it. No user action is required.

2.1.1 Installing on Linux

Several formats are available for Linux. The packages and archives below **make blunderDB executable automatically**: unlike the raw binary downloaded through a browser, they avoid having to run `chmod +x` after every download or update. They also add an entry to the applications menu.

Native packages (.deb / .rpm)

Recommended method on Debian, Ubuntu and Linux Mint (`.deb`) as well as Fedora and openSUSE (`.rpm`). The package manager automatically installs the appropriate webkit2gtk dependency. Replace `x.y.z` with the downloaded version:

```
sudo apt install ./blunderdb_x.y.z_amd64.deb      # Debian / Ubuntu / Mint
sudo dnf install ./blunderdb-x.y.z.x86_64.rpm     # Fedora / openSUSE
```

Arch Linux (AUR)

The `blunderdb-bin` package is available on the AUR and updated automatically by AUR helpers:

```
yay -S blunderdb-bin      # ou : paru -S blunderdb-bin
```

.tar.gz archive

For other distributions. Extracting an archive preserves the executable bit, so no `chmod` is needed:

```
tar xzf blunderDB-linux-webkit2gtk-4.1-x.y.z.tar.gz
cd blunderDB-linux-webkit2gtk-4.1-x.y.z
./blunderdb
```

Raw binary (advanced method)

The raw binary https://github.com/kevung/blunderDB/releases/latest/download/blunderDB-linux-0.32.0 remains available. Since a browser strips the executable bit on download, you must restore it before the first run:

```
chmod +x ./blunderDB-linux-x.y.z
```

Note

Two Linux variants are published depending on the webkit2gtk library version. If you get the error `libwebkit2gtk-4.0.so.37: cannot open shared object file`, your distribution uses webkit2gtk-4.1: use the `.deb`/`.rpm` package, the AUR package, or download the dedicated build https://github.com/kevung/blunderDB/releases/latest/download/blunderDB-linux-webkit2gtk-4.1-0.32.0. Native packages select the correct dependency automatically.

2.1.2 Windows and Mac warnings

Warning

On Windows, it is possible that it may hesitate to run blunderDB. See: Section 2.10 for an explanation of why and how to bypass any potential blocks.

Warning

On Mac, it is possible that it may have reservations about running blunderDB. See Section 2.11 to understand why and bypass any potential blocks.

2.2 Manual

2.2.1 Introduction

blunderDB is software for creating backgammon position databases. Its main strength is to provide a single place to aggregate positions that a player has encountered (online, in tournaments) and to be able to re-study these positions by filtering them according to various arbitrarily combinable filters. blunderDB can also be used to create catalogs of reference positions.

Positions are stored in a database represented by a *.db* file.

2.2.2 Main Interactions

The main interactions possible with blunderDB are:

- add a new position,
- modify an existing position,
- copy the board image to the clipboard (PNG) via **Ctrl+X**, or with the full analysis via **Ctrl+X Ctrl+X**,
- delete an existing position,
- search for one or more positions,
- import matches from various sources (XG, GNUbg, BGBlitz, Jellyfish), including comments from XG files,
- navigate through the moves of an imported match,
- organize positions into collections,
- organize matches into tournaments.

The user can freely tag positions and annotate them with comments.

2.2.3 Description of the interface

The interface of blunderDB is composed, from top to bottom, of:

- [top] the toolbar, which gathers all the main operations that can be performed on the database,
- [in the middle] the main display area, which allows for displaying or editing backgammon positions,
- [at the bottom] the status bar, which provides various information about the database or the current position, and integrates the command line.

Panels can be displayed to:

- display the analysis data associated with the current position from eXtreme Gammon (XG), GNUbg, or BGBlitz,
- display, add, or modify comments,
- search and filter positions using combinable criteria,
- display and manage position collections (collections panel),
- display the list of imported matches and navigate through the moves of a match (match panel),
- display and manage tournaments (tournaments panel),
- display performance statistics (Stats panel),
- compute the EPC (Effective Pip Count) of a bearoff position (Bearoff panel),
- study positions with spaced repetition (Anki panel),
- display the database metadata (metadata panel).

Modal windows can be displayed to:

- display the blunderDB help,
- display the catalogue of guided tours (see *Guided tours and sample database*),
- configure database export settings,
- configure blunderDB, in particular the interface language (see *Configuration*).

The main display area provides the user with:

- a board to display or edit a backgammon position,

- the level and owner of the cube,
- the pip count of each player,
- the score of each player,
- the dice to play. If no values are shown on the dice, the position of the dice indicates which player is on roll and that the position is a cube decision. When the cube decision is a response to a double (take/pass), the offered cube is shown in the centre of the board, at the offered value.

The status bar is structured from left to right with the following information:

- the command line, accessible by pressing the *SPACE* key,
- an informational message related to an operation performed by the user,
- the index of the current position, followed by the number of positions in the current library (or move/game info when navigating a match).

Note

In the case of positions resulting from a user search, the number of positions indicated in the status bar corresponds to the number of filtered positions.

2.2.4 View Tabs

Below the toolbar, a tab bar lets you work with several **views** in parallel. Each view is an independent workspace that keeps its own position list, the index of the current position, the displayed position, the analysis and the selected move, the active panel, the comment being edited, as well as the navigation context within a match. This makes it possible, for example, to keep a search open in one view while browsing a match in another.

- **Create a view**: click the + button of the tab bar or press *CTRL-T*. The new view starts as a copy of the current view.
- **Close a view**: click the cross on the tab or press *CTRL-W*. The last view cannot be closed.
- **Switch view**: click a tab, press *CTRL-PageUp* / *CTRL-PageDown* (or *SHIFT-J* / *SHIFT-K*) to move to the previous / next view, or *CTRL-1* to *CTRL-9* to jump directly to the n-th view.
- **Rename a view**: double-click the tab, type the new name and confirm with *ENTER*.

Views are saved with the database session state and restored when it is reopened.

2.2.5 Configuration

The configuration button (gear-shaped icon) located in the toolbar, to the left of the help button, opens the blunderDB settings window. It is organised into three tabs:

- **Interface** — language, display scale, panel position;
- **Colours** — the board's colours;
- **Issuer identity** — the key that signs your watermarks, described in *Handing out a database: origin and password*.

The *Interface* tab lets you choose the language among English, French, German, Italian, Spanish, Finnish, Japanese, Greek and Russian. The whole interface (toolbar, panels, messages, help) is translated into the selected language. The language choice is saved and kept across sessions.

The *Board colours* tab lets you customise the board's colours. Each element has its own colour picker: the background, the border, the light and dark points, player 1's and player 2's checkers, the dice, the dice pips and the cube. The *Reset* button restores all the default colours. Like the language, the chosen colours are kept across sessions.

The configuration window also provides interface display settings. An **interface scale** slider lets you enlarge or shrink all interface elements, which is useful on high-density screens or to improve readability. A **panel position** menu sets where the panels (search, matches, analysis) appear relative to the board: *bottom*, *side* or *automatic* (the side is then chosen on wide screens to make better use of the available space). Like the other settings, these choices are kept from one session to the next.

2.2.6 Guided tours and sample database

To make getting started easier, blunderDB offers **guided tours** of the interface. The tour catalogue opens from the toolbar or with the `tour` command (alias `tutorial`). Four tours are available: a general tour of the interface, and tours dedicated to searching positions, reviewing matches and reviewing tournaments. Each tour highlights the relevant interface elements, step by step, and can be replayed at any time. On first launch, the general tour is offered automatically.

The `demo` command loads a **sample database** (matches, a tournament and analyses) that lets you explore the tool's features without importing your own games. The guided tours rely on this database when no database is open.

2.2.7 Browsing positions

By default, blunderDB allows you to:

- scrolling through the different positions in the current library,
- displaying the analysis information associated with a position.
- displaying, adding, and modifying comments on a position.

Tip

Refer to *Keyboard shortcuts* for available shortcuts.

2.2.8 Editing positions

Pressing the *TAB* key opens the search panel and allows editing a position on the board to add it to the database or to define a position structure to search for. The distribution of checkers, the cube, the score, and the turn can be modified using the mouse (see *Edit a position*).

Tip

Refer to *Keyboard shortcuts* for available shortcuts.

2.2.9 The command line

The command line, integrated into the status bar, allows you to perform all the functionalities of blunderDB available in the graphical interface: general operations on the database, position navigation, displaying analysis and/or comments, searching for positions based on filters… After getting familiar with the interface, it is recommended to gradually use the command line for a powerful and smooth use of blunderDB, especially for position search functionalities.

To open the command line, press the *SPACE* key. To submit a query and close the command line, press the *ENTER* key.

blunderDB executes the queries sent by the user as long as they are valid and immediately modifies the state of the database if necessary. There are no explicit save actions required from the user.

Tip

Refer to Section 2.4 for the list of available commands in the command line.

2.2.10 Analysis Panel

The **Analysis** panel (*CTRL-L*) displays the analysis data for the current position, imported from eXtreme Gammon (XG), GNUbg, or BGBlitz. It shows the best alternatives (checker moves or cube decisions) with their equity values and corresponding errors. The *d* key toggles between checker and cube analysis. During match navigation, the actually played move is highlighted in the list of alternatives. Press *CTRL-L* or run the `list` command to show or hide the panel.

2.2.11 Comments Panel

The **Comments** panel (*CTRL-P*) displays, adds, and edits comments associated with the current position. Comments imported from XG files are automatically associated with the corresponding positions. Press *CTRL-P* or run the `comment` command to show or hide the panel.

2.2.12 Search Panel

The **Search** panel (*CTRL-F* or *TAB*) filters positions using freely combinable criteria: checker structure, cube decision type, error magnitude, dates, tags, etc. The *TAB* key simultaneously opens the search panel and the position editor, allowing a checker structure to be defined directly on the board.

To refine a search among the currently filtered positions, use the `ss` command followed by filters (e.g.: `ss nc`, `ss E>40`). The search panel also offers a *Search in current results* checkbox for the same functionality.

The panel offers an explicit control over the **decision type** searched for: *Indifferent* (no filter), *Checker* (checker decisions) or *Cube* (cube decisions). When *Cube* is selected, a second list specifies the sub-type: *All*, *Double / No double* (the player on roll has to decide whether to double) or *Take / Pass* (response to an opponent's double). The control is synchronised with the board: editing the dice or the cube on the board updates the decision type, and vice versa. In *Take / Pass* mode, the cube is shown in the centre of the board at the offered value; that value remains editable.

The **Flagged** filter keeps the positions you flagged in the software the match came from. Only eXtreme Gammon produces this information, recorded move by move in the `.xg` file; blunderDB reads it on import and keeps it. A flagged cube decision yields two flagged positions, the double and the take/pass, blunderDB splitting in two what the source file records as a single decision.

Note

Flagging is not retroactive: matches already in the database do not carry this information, since it exists only in the source files. Simply import the relevant `.xg` file again — the import detects the duplicate and adds nothing but the flags, leaving existing comments and analyses untouched. A flag can neither be set nor removed from within blunderDB: for a temporary working list, use a collection instead.

The **Comment** filter queries the comments attached to positions in three exclusive modes. *contains text* searches for one or more words in the comment text (input field, words separated by `;`, at least one must match); *has a comment* keeps any position carrying a comment, whatever its content; *no comment* keeps, on the contrary, the positions that are not annotated — useful, combined with an error or date filter, to draw up the list of what remains to be commented.

Note

Comments imported from a match file (XG, GNUbg) count as comments: blunderDB does not record their origin and therefore cannot tell a note you typed from one taken over from the source file. Moreover, comments attached to

a *match* or a *tournament* are not concerned: they annotate the match or the tournament, not its positions.

The **Matches & Tournaments** filter is backed by a shared picker (a modal window) instead of typed numeric IDs: two checkbox lists, one for matches and one for tournaments, each text-filterable (player, date, event for matches; name, date, location for tournaments), with *All* / *None* buttons that act only on the currently filtered subset. Checking a tournament automatically checks (and greys out) its member matches in the match list, making visible the fact that a tournament is equivalent to the set of its matches.

The search panel has three tabs along its left edge: *Search* (the filters), *History* and *Saved*. The **History** tab lists past searches with their date and command: a click selects a search and displays the associated position on the board, a double-click re-runs it. Each entry can be saved to the filter library (bookmark icon, by giving the filter a name) or deleted. The **Saved** tab contains the **filter library**: double-click a saved filter to re-run the corresponding search (see *Annex: Advanced Filter Usage*). The `history` command (alias `hi`) opens the search panel.

Tip

Refer to Section 2.4 for the list of available filters.

2.2.13 Collections Panel

The **Collections** panel (*CTRL-B*) manages collections of positions. Collections can be created, renamed, and deleted. Positions can be added to or removed from them. Double-click a collection to browse its positions using the *LEFT* and *RIGHT* keys. The order of collections and positions within them can be changed by drag and drop. Press *CTRL-B* or run the `collection` command to show or hide the panel.

2.2.14 Matches Panel

The **Matches** panel (*CTRL-Tab*) lists imported matches. Double-click a match (or press *ENTER*) to navigate through its moves. The `m` command resumes navigation in the last visited match.

The user can:

- browse through the moves of a match using the *LEFT* and *RIGHT* keys,
- switch between games using the *PageUp* and *PageDown* keys,
- display the move analysis (checker and cube) by pressing *CTRL-L*,
- toggle between checker move and cube analysis with the *d* key,
- see the actually played move highlighted in the analysis.

The last visited position in each match is saved and restored automatically. Press *CTRL-Tab* or run the `match` command to show or hide the panel.

Each match can be exported as a Jellyfish `.mat` transcript via the ⬇ button in the match list or the *.mat* button of the match sheet.

The **Merge players** button in the panel toolbar opens a window listing all the player names in the database with their number of matches: select the spelling variants of the same player, choose the canonical name to keep, then merge. Useful to unify per-player statistics when the same player appears under several names.

When a match is open, an **information bar** appears above the board: it recalls the players involved (*player 1* versus *player 2*) as well as the match context (event, location, round, date and match length, when this information is available). This bar is also shown outside match mode: when a studied position (from a search, a collection or a direct access) comes from one or several matches, it indicates its **provenance** — the first match concerned and, where applicable, a "+N" badge listing the others on hover. A position imported on its own, which no match references, shows nothing.

When opening a database that contains matches, the **Matches** panel is shown right away and the review starts directly on the first position, so you can begin navigating immediately.

Note

A database can be opened for writing by only one window at a time. If you open a database already open in another blunderDB window, it opens **read-only**: navigation, search and analysis remain possible, but any modification is disabled and the title bar shows "[read-only]".

Tip

Refer to *Keyboard shortcuts* for available shortcuts.

2.2.15 Tournaments Panel

The **Tournaments** panel (*CTRL-Y*) groups matches into tournaments for organised tracking and per-event statistical analysis. Tournaments can be created, renamed, and deleted; matches can be assigned to them. Stats panel statistics can be filtered by tournament. Press *CTRL-Y* to show or hide the panel.

The **PR** column of each tournament shows the PR of the **reference player** — that is, the player appearing in the greatest number of the tournament's matches (in case of a tie, the one who made the most decisions). The PR therefore does not mix your play with your opponents': for your own tournaments, it reflects your performance alone. The reference player's name appears in a tooltip when hovering over the value.

2.2.16 Stats Panel

Introduction

The **Stats** panel lets you analyse your play level and track your progress over time using the positions imported in the database. It computes and displays **PR** (Performance Rate) and **MWC cost** (Match Winning Chance cost) for all positions or a filtered subset.

The Stats panel is especially useful for:

- **gauging your level** against reference thresholds (world-class, expert, advanced…) using the global PR;
- **tracking your progress** tournament by tournament or match by match using the Progression tab charts;
- **identifying your weak spots**: the Errors tab shows the breakdown between checker plays and cube decisions, and the distribution of error magnitudes;
- **navigating directly to the relevant positions** by clicking any indicator (drill-down).

Opening the panel

To open the Stats panel:

- Press *CTRL-D*.
- Type the command `:stats` or `:st` in the command line.

Note

The panel refreshes automatically whenever the filter is changed. It does not recalculate statistics on a simple PR ↔ MWC toggle: both metrics are computed simultaneously by the backend.

Filter bar

The filter bar at the top of the panel restricts the computation to a subset of positions.

Player perspective

The **Player** drop-down filters statistics to the analysed player. blunderDB automatically selects the player whose name appears most often in the database — changeable at any time.

> **Tip**
>
> Changing the player does not cause any data loss; simply re-select the previous player in the list.

Available filters

- **Tournament(s)** — restrict to one or more tournaments. Multiple tournaments can be selected simultaneously.
- **Dates** — time range (*From* … *To*). If only the start date is set, more recent positions are included.
- **Decision type** — All / Checker plays / Cube decisions.
- **Match length** — restrict to specific match lengths (1, 3, 5, 7, 9, 11, 13, 15, 21 points). Multiple lengths can be combined.

A **Reset** button clears all filters (except the auto-detected player).

> **Note**
>
> Filters are saved in the blunderDB configuration (`config.yaml`) and restored on the next launch.

PR / MWC toggle

The **PR / MWC** button at the top of the panel toggles the metric displayed across all tabs.

PR (Performance Rate)

Measures *money-game* play quality: sum of errors in milli-points, divided by the number of decisions. Independent of match score.

Approximate reference thresholds:

Level	PR
World-class	< 3
Expert	3 – 5
Advanced	5 – 8
Intermediate	8 – 12
Beginner	> 12

MWC cost (Match Winning Chance cost)

Cumulative match winning probability lost due to errors, over the full filtered dataset. Computed using the Kazaross-XG2 MET embedded in blunderDB.

> **Caution**
>
> MWC cost **does not apply** to *money-game* positions (with no match stake). Those positions are excluded from the MWC computation. MWC values depend on the MET used; they are not directly comparable across software using different METs.

The PR ↔ MWC toggle is instant: no backend recalculation is performed.

Dashboard tab

The **Dashboard** tab gives a summary view of key indicators.

Level cards

Three cards display the PR (or MWC) for:

- **All** — all decisions (checker + cube);
- **Checker** — checker plays only;
- **Cube** — cube decisions only.

Clicking a card loads the positions in the corresponding subset into the analysis panel (drill-down).

> **Note**
>
> The total number of decisions is shown at the bottom of each card on hover.

Rolling PR over last N decisions

A row of PR (or MWC) values computed over the last *N* decisions (N = 5, 10, 50, 100, 250, 500, 1000) lets you measure the recent trend. Greyed values correspond to an N larger than the number of available decisions.

Clicking a value loads the corresponding last *N* positions.

Top blunders

The list of the 10 worst errors (or MWC cost), sorted by descending magnitude. Clicking a row loads the relevant position in the analysis panel.

Progression tab

The **Progression** tab shows how your level evolves over time.

Tournament line chart

A line chart displays the PR (or MWC) for each tournament (X axis: tournament order, Y axis: metric value). Colour bands materialise the level thresholds.

Clicking a point on the chart opens a context menu with two options:

- **Open tournament** — opens the tournament in the Tournaments panel.
- **Open positions** — loads all positions from the tournament into the analysis panel.

Match scatter plot

A scatter plot represents each match (X axis: date, Y axis: PR or MWC). Point size is proportional to the number of decisions in the match.

Clicking a point opens a context menu:

- **Open match** — opens the match in the Matches panel.
- **Open positions** — loads all positions from the match into the analysis panel.

Errors tab

The **Errors** tab breaks down error sources.

Breakdown by cube action

A bar chart displays the PR (or MWC) for each type of cube decision: *NoDouble*, *DoubleTake*, *DoublePass*, *TooGood*. Each bar also shows the number of decisions and the blunder rate in a tooltip.

Clicking a bar loads the positions matching that cube action, **only those with an error** (drill-down).

Checker / Cube comparison

A comparison chart places checker plays and cube decisions side by side. Clicking a bar loads the positions in the subset with an error.

Error magnitude histogram

A histogram distributes errors by magnitude in milli-points (buckets: 0–5, 5–10, 10–25, 25–50, 50–100, ≥ 100). Clicking a bar loads the positions in the bucket.

Aggregation rule

Important

The PR of a tournament (or any subset) is computed using the **sum/sum** rule — never as an average of individual match PRs.

Formula:

$$PR_{tournament} = \frac{\sum_i \text{error}_i}{\text{total number of decisions}}$$

Example: a player plays two matches in a tournament —

- Match A: 10 decisions, total error 50 mp → PR = 5.0
- Match B: 90 decisions, total error 270 mp → PR = 3.0

Naive average of PRs: (5.0 + 3.0) / 2 = **4.0** *(incorrect)*

Sum/sum rule: (50 + 270) / (10 + 90) = 320 / 100 = **3.2** *(correct)*

The sum/sum rule is the only one that handles varying match lengths correctly (a 21-point match carries more weight than a 1-point match).

MWC: limitations

- The MWC cost is computed using the **Kazaross-XG2 MET**, the de facto reference table in competitive backgammon. Results are not directly comparable with software using other METs.
- *Money-game* positions (with no match score) are **excluded** from the MWC computation. If your database contains many money-game positions, the MWC cost may be underestimated or unavailable.
- The MWC cost is cumulative over the full filtered dataset — not a per-decision indicator. It measures the total impact of your errors on your winning chances.

2.2.17 Bearoff Panel

The **EPC** panel (*CTRL-E*) computes the EPC (Effective Pip Count) of a bearoff position. It is activated by pressing *CTRL-E*, clicking the EPC tab in the bottom panel, or running the `epc` command.

In this panel, the user edits the checker positions in the two home boards: clicking points 1 to 6 places checkers of the bottom player, clicking points 19 to 24 places checkers of the top player (either mouse button works).

Results are presented as two stacked tables:

- at the top, a **players table** — one row per player (identified by its coloured dot, the black player always at the bottom), one column per quantity: EPC, pip count, wastage (difference between the EPC and the pip count), average number of rolls and standard deviation. When both players have checkers in their home board, a **Δ** row gives the *signed* differences (bottom − top: negative when the black player is ahead) in EPC, pips and wastage;
- below it, separated by a rule, a **race/cube table**: two winning-probability columns (one per player dot, values without the % sign) and the money equities — under each decision other than the best one, the equity gap to the best decision is shown in parentheses. The **decision** is displayed to the right of the table, with the exact/estimated badge. The **player on roll** and the **cube position** are edited directly on the board, as in edit mode: clicking a player's bearoff/score rectangle gives that player the roll; clicking the cube cycles centred → owned bottom → owned top (right click cycles the other way). The cube value stays pinned — in money game the equities are expressed in units of the current cube, only its owner matters. The analysis is recomputed immediately. In the estimated regime, the "See the configuration settings" link opens the *Bearoff* tab of the configuration directly.

The *Challenge* box stays pinned at the top right of the panel.

When the position is a pure bearoff (all checkers of both players in their home board), the race table shows, for the player on roll:

- the winning probability,
- in the *exact* regime: the money equities (cubeless, no double, double/take, double/pass) and the **money cube verdict** (no double, double/take or double/pass),
- in the *estimated* regime: the winning probability alone, together with its error margin — the cube verdict is then deliberately not shown.

A badge indicates the regime: **exact** (value read from a two-sided database) or **estimated ± margin**. See *Methodology and assumptions of the Bearoff panel* for the precise definition of the two regimes and their assumptions.

Widening the exact domain. The built-in database covers 6 checkers per player. Two ways to go beyond that, in the *Bearoff* tab of the configuration:

- download the extended TS-06-11 database (1.2 GB, verified by SHA-256, deletable in the same place): exact verdict up to 11 checkers per player. An interrupted or cancelled download **resumes where it stopped** (the partial file is kept and completed with an HTTP Range request);
- point to any two-sided gnubg `.bd` file. The database with the widest domain automatically wins.

Challenge mode. The panel's *Challenge* box enables a training mode: on every change of the position, the values of the three areas — the bottom player's row, the top player's row and the race table — are hidden (replaced by "···"); clicking an

area reveals that area only. The Δ row only appears once both player rows are revealed. You can thus practise estimating each side's EPC, then committing to a cube decision, before checking. The setting is remembered.

To close the Bearoff panel, press *CTRL-E* or switch to another tab.

Methodology and assumptions of the Bearoff panel

Every value displayed by the panel rests on precise assumptions, stated here exhaustively.

Domain. The panel only handles pure bearoff: all remaining checkers of both players in their home board. The position is evaluated *before the roll*; any dice already showing are ignored. No-contact races with checkers outside the home board are not handled.

EPC blocks (always exact). The EPC, the average number of rolls and the standard deviation come from the exact distribution of the number of rolls needed to bear off all checkers, read from GNUbg's one-sided database (6 points, 15 checkers, built in). EPC = average rolls × 49/6 (49/6 ≈ 8.167 is the exact average number of pips per roll, doubles counted four times); wastage = EPC − pip count. The only idealisation is *optimal one-sided play*: each player minimises their own rolls while ignoring the opponent — this is the standard definition of the EPC.

Winning probability, exact regime. Direct lookup in the widest available two-sided database (built-in TS-06-06, external file, or downloaded TS-06-11). These databases result from a complete retrograde analysis under optimal two-sided play by both sides: no additional assumption, error limited to quantisation (< 0.002%).

Winning probability, estimated regime. Outside the database's domain: the probability is obtained by convolving the two one-sided distributions (the player on roll wins if their number of rolls is less than or equal to the opponent's), then applying a frozen polynomial correction, calibrated offline against the TS-06-11 database. Three assumptions:

- **independence** of the two bearoff processes — structural in a race, with no contact there is no interaction whatsoever;
- **optimal one-sided play by both sides** — this is *the approximation*: in reality the trailing player deviates to play for variance and the leader for safety. The measured effect is an antisymmetric bias (the convolution overstates the leader's advantage) which the correction absorbs statistically;
- the **correction** was calibrated and validated on the oracle's domain (up to 11 checkers per player). Measured residual error: standard deviation 0.05%, 99th percentile 0.17%, observed maximum 0.9% (in winning-probability points). **Beyond 11 checkers per player, this bound is extrapolated** — the trend is monotonic but no oracle certifies it.

Equities and cube verdict (exact regime only). The displayed equities are those of the **money game, without Jacoby**, the reference framework of the bearoff literature. Within the ≤ 11 checkers per player domain, gammons are impossible (each side has already borne off at least 4 checkers): this is not an approximation. The verdict (no double / double, take / double, pass) is reconstructed exactly from the stored equities, following GNUbg's rule, validated verdict for verdict against its analysis.

Note

The cubeful equities assume **optimal cube play by both sides all the way to the end**: future recubes are fully valued (complete retrograde analysis). In the very volatile races at the end of the game, the cascade of recubes eats up almost all of the advantage of the side on roll — the "no double" and "double/take" equities can then be close to zero where an engine such as XG, whose cube model does not value this cascade, shows values close to the dead cube (for instance 2 checkers on the 3 point against 2 checkers on the 2 point: 62% winning chances, exact D/T +0.006 versus +0.475 for XG). The displayed **decision**, however, coincides with the engines'.

Why is the verdict never estimated? Cubeful equity is a *trajectory* problem (when to double) that no statistical summary of the position captures: the best static model measured leaves a residual error (standard deviation 0.016 of equity, maximum 0.20) large enough to flip every close decision. Likewise, converting the verdict to the match score through a match-equity table was measured to be insufficient (12% of disagreements with GNUbg's 2-ply analysis, including genuine

blunders). Since a wrong verdict displayed with confidence is worse than no verdict, blunderDB only shows the verdict when it is exact — and money.

Note

Bearoff databases are immutable mathematical tables, regenerable with GNUbg's `makebearoff` tool.

2.2.18 Anki Panel

The **Anki** panel (*CTRL-K*) allows studying positions with spaced repetition using the FSRS algorithm. Users can create decks from collections or search results.

Creating decks: Click *New Deck* to create a deck from a collection or the current search results. Search-based decks sync automatically when the Anki tab is opened.

Reviewing: Select a deck then click *Study* (or double-click a deck) to start reviewing due cards. Each card shows the corresponding position on the board. Rate your recall with keys *1* (Again), *2* (Hard), *3* (Good), or *4* (Easy). Press *Esc* to stop and return to the deck list.

Free drill (cram): The *Cram* button, next to *Study*, starts a free-drill session: random positions from the deck are shown to you regardless of the FSRS schedule. This mode **never alters the spaced-repetition plan** — ideal for warming up before a tournament or intensively reviewing a themed deck without disturbing its ordering. A *Cram* badge replaces the card state and a *Next* button (keys *1* to *4*) cycles through the positions. *Esc* returns to the list without saving an interrupted session.

Pause/Resume: You can interrupt a review session at any time with *Esc*. The button changes to *Resume* and shows your progress. Click it to pick up where you left off.

Deck management: Use the action buttons to rename, sync, reset, or delete decks. FSRS parameters (target retention, maximum interval, fuzz factor) can be configured per deck in the Settings (gear icon).

2.2.19 Metadata Panel

The **Metadata** panel displays general information about the current database: name, description, number of positions, matches and games, schema version. Accessible via the `meta` command.

It also shows the database's origin **when there is one** — see *Handing out a database: origin and password*. An ordinary database does not show that section.

2.2.20 Handing out a database: origin and password

A teacher handing out a database of positions has two mechanisms, independent of each other, both optional and both chosen **at export time**: marking the file with its origin, and protecting it with a password.

Note

Neither tracks what becomes of the file. blunderDB **records nothing on the recipient's side**: opening a marked database is exactly like opening any other, and nothing anywhere logs who opened it, when, or where its contents came from.

Marking a database with its origin

The export dialog fits on a single screen: the form, then a progress overlay laid over it while the file is written. It closes by itself when finished, and the result appears in the status bar.

Three points deserve attention:

- **The export covers the positions currently displayed**, not the whole database. After a search, only the results go out — the dialog says so at the top.
- **A collection whose positions are not all in the selection arrives truncated.** The list therefore shows, for each collection, how much of it is covered ("12/40"), in red when it is partial.
- **Tournaments can only be exported together with matches**: without them the tournament–match link does not exist and the tournament would arrive empty. The box stays disabled until "include matches" is ticked.

The *User*, *Description* and *Date* fields describe the **file being produced**; they are pre-filled from the source database. The *My saved filters* box is kept apart from the others: it exports not content but your own saved searches, which are of no use in someone else's database.

Ticking **Mark this file with its origin** reveals two fields:

- **Origin** — what this file is and where it comes from, in your own words: "Jean Dupont's lesson — 12 March 2026". This field is **required**: while it is empty the export button stays disabled.
- **Note**, optional — terms of use, a contact address, a request not to pass the file on.

The mark is signed with your issuer identity. It is therefore **tamper-evident and unforgeable**: nobody can alter it, nor fabricate one in your name. It is however **not unremovable** — the distributed file is an ordinary SQLite database, and blunderDB is free software. It prevents nothing: it says where the file came from.

Issuer identity

Marks are signed with your **issuer identity**, created by itself the first time you mark a file; there is nothing to set up. It belongs to a person rather than to a database: every file you mark carries the same public fingerprint, of the form `A3F1-9C24-7B05-E1D8`.

You can give that fingerprint to your recipients so they can check that a file really comes from you. The identity moves from one machine to another as a single file (extension `.bdbid`), optionally protected by a passphrase. **That file lets anyone holding it sign in your name: do not share it.**

In the preferences (the gear icon in the toolbar), the *Issuer identity* tab shows your name and fingerprint, and offers *Save identity…*, *Load identity…* and *Regenerate…*.

Warning

Regenerating revokes nothing. A watermark embeds the public key that signed it, so it verifies for ever, on its own. If your identity file has leaked, whoever holds it can keep signing under your old fingerprint, and those marks stay valid.

What protects you after a leak is not software: it is publishing your new fingerprint and disowning the old one to your recipients.

Regenerating overwrites the current key; blunderDB offers to save it before replacing it.

Protecting a database with a password

The password is typed masked, here as when opening a protected file; the eye icon reveals it **while it is held down**, and masks it again as soon as it is released.

Ticking **Protect this file with a password** produces a file with the `.dbx` extension — even if you chose a `.db` name in the save dialog, which opens before the password is asked for. To open it, use the usual open-database action: the file chooser accepts both `.db` and `.dbx`. blunderDB then asks for the password and installs an ordinary database beside it; nothing is asked afterwards.

The dialog offers to **delete the protected file once opened**: without that you keep the same content under two names. The box is not ticked by default — the protected file is yours to keep if you mean to pass it on — and the deletion only happens after a successful open.

Warning

The password protects the file **in transit**, not the database. It stops a stranger opening a file left in a downloads folder or an attachment forwarded by mistake. It does not protect you from whoever you gave the password to.

The password is checked on **every** open, including when the file has already been opened on this machine before.

Technically, the database is encrypted with **AES-256 in GCM mode**, with the key derived from the password by **Argon2id** (64 MiB of memory, 3 passes, 4 lanes) and a random salt unique to each file. GCM authenticates the whole payload: a wrong password is detected as such, and so is any tampering with the encrypted file — you never silently end up with a corrupt database.

The protected file's header stays **in the clear**: its origin remains readable without the password.

Reading a file's origin

In the application, open the file and show the **Metadata** panel (the `meta` command). An **Origin** section appears at the top of the panel, read-only, stating what was written, by whom, when, and how the signature checks out:

- "✓ signature verified — marked by you": the file carries your mark, intact;
- "✓ signature verified": the mark is intact and comes from another key — compare its fingerprint with the one the producer gave you;
- "⚠ invalid signature": the document has been altered or forged.

This section does not appear on an ordinary database.

From the command line, `blunderdb info --db file.db` shows the origin and the state of the signature, **without ever writing to the file**. It works on a protected file too, without the password. See `CLI_USAGE.md` for `export`'s `--watermark` and `--password` options, and for `identity` and `open`.

2.3 User Guide

This guide is a practical introduction to pick up quickly blunderDB.

2.3.1 Create a new database

To create a new database, click on the "New Database" button in the toolbar. Choose a path to save the database, as well as a name, and click "Save".

Note

The file extension for blunderDB databases is *.db*.

Tip

Keyboard shortcuts: *CTRL-N*. Command: `n`

2.3.2 Open an existing database

To load an existing database, click on the "Open Database" button in the toolbar. Navigate to the path where the database is located, select the *.db* file, and click "Open."

Tip

Keyboard shortcuts: *CTRL-O*. Command: ○

2.3.3 Import and merge a database

To import and merge another blunderDB database into the currently open database, click on the "Import Database" button in the toolbar. Select the *.db* file to import and click "Open."

blunderDB will intelligently merge the two databases:

- Positions that do not exist in the current database will be added with their analyses and comments.
- Positions that already exist will be updated: analyses will be completed if missing, and comments will be merged (adding new comments without duplicating existing ones).
- A message will summarize the number of positions added, merged, and ignored.

Note

The import requires that both databases have compatible schema versions. It is possible to import a database with a lower or equal version into a database with a higher version.

Caution

The import operation immediately modifies the currently open database. It is recommended to make a backup copy before importing another database.

2.3.4 Edit a position

To edit a position, press the *TAB* key to open the search panel and the position editor. Edit the position with the mouse:

- Click on the points to add checkers. A left-click assigns checkers to player 1. A right-click assigns checkers to player 2. To insert a prime, click on the starting point, hold down the button, and release on the endpoint. Click on the bar to place checkers in the bar.
- To clear the position, double-click on an empty area outside the board or press the *BACKSPACE* key.
- To send the cube to Player 1, left-click on the cube. To send the cube to Player 2, right-click on the cube.
- To indicate the player who is to move, click on the designated dice area.
- To edit the dice, left-click to increase the value of a die, right-click to decrease the value of a die. If the dice faces are empty, it means the position is a cube decision.
- To edit the players' score, left-click to increase the score, right-click to decrease the score.

Tip

The input of the position with the mouse for the checkers is done in the same way as in XG.

2.3.5 Add a position to the database

After editing the position, the search panel is open.

To save the previously obtained position, press *CTRL-S* or click the "Save Position" button in the toolbar.

Tip

Open the command line and execute: `w`

2.3.6 Tag a position

To add a tag *toto* to the current position, open the command line by pressing *SPACE*, type `#toto`, and confirm the command by pressing *ENTER*.

2.3.7 Delete a position

To delete the current position from the database, press *Del* or click the "Delete Position" button in the toolbar.

Tip

In the command line, execute `d`.

Caution

The deletion of the position is final and does not require any user confirmation.

2.3.8 Import a position from XG

To import a position directly from XG,

1. display the position to import in XG and press *CTRL-C*,
2. open blunderDB and press *CTRL-V*.

Note

The automatic paste detects the source format (XG, GNUbg, BGBlitz).

2.3.9 Import a match

blunderDB can import matches from various sources.

Supported formats:

- eXtreme Gammon (XG): *.xg* and *.xgp* (positions) files
- GNUbg: *.sgf* files
- Jellyfish: *.mat* and *.txt* files
- BGBlitz: *.bgf* and *.txt* files

To import one or more match files:

1. Press *CTRL-I* or click the "Import" button in the toolbar.
2. Select one or more files to import.
3. blunderDB automatically detects the format and imports the match.
4. A progress window shows the number of imported, failed, and skipped (duplicate) files.

Tip

Command: `i`

Note

blunderDB automatically detects duplicates and prevents importing a match already present in the database.

2.3.10 Import a folder of matches

To recursively import all match files contained in a folder and its subfolders:

1. Press *CTRL-SHIFT-F* or click the corresponding button in the toolbar.
2. Select the folder containing the match files.
3. blunderDB automatically collects and imports all recognized files (*.xg*, *.xgp*, *.sgf*, *.mat*, *.txt*, *.bgf*).

2.3.11 Drag and drop

blunderDB supports drag and drop. You can drag and drop onto the blunderDB window:

- match or position files (*.xg*, *.xgp*, *.sgf*, *.mat*, *.txt*, *.bgf*) to import them,
- database files (*.db*) to open or merge them with the current database,
- folders to recursively import all the files they contain.

2.3.12 Navigate through a match

To navigate through an imported match:

1. Open the match panel with *CTRL-Tab*.
2. Double-click on a match or press *ENTER*.
3. Use the *LEFT* / *RIGHT* keys to browse through moves.
4. Use *PageUp* / *PageDown* to switch between games.
5. Press *CTRL-L* to display the analysis.
6. Press *d* to toggle between checker move analysis and cube analysis.

Tip

Shortcut: *CTRL-Tab* to open the match panel. Command: `m`

Note

blunderDB remembers the last visited position in each match. When returning to a match, the last visited position is automatically restored.

2.3.13 Manage the match panel

The match panel (*CTRL-Tab*) allows you to:

- list all imported matches (sorted from most recent to oldest),
- sort matches by column (player 1, player 2, date, match length, tournament),
- edit player names or date by double-clicking on the fields,
- swap player 1 and player 2 using the swap button,
- assign a match to a tournament,
- delete a match using the *Del* key.

2.3.14 Manage collections

Collections allow you to organize positions into custom groups. To access the collections panel, press *CTRL-B*.

Create a collection:

1. Open the collections panel (*CTRL-B*).
2. Enter the name of the new collection and click “Add”.

Add positions to a collection:

1. Select the desired positions.
2. Add them to the collection from the collections panel.

Browse a collection:

- Double-click on a collection to browse its positions. The order of collections and positions can be changed by drag and drop.

Tip

Command: `coll`

2.3.15 Manage tournaments

Tournaments allow you to organize imported matches by event. To access the tournaments panel, press *CTRL-Y*.

Create a tournament:

1. Open the tournaments panel (*CTRL-Y*).
2. Click “Add” and enter the tournament name.

Assign a match to a tournament:

- From the match panel (*CTRL-Tab*), use the dropdown menu in the tournament column to assign a match.

2.3.16 Display performance statistics

The Stats panel lets you view your performance indicators (PR and MWC cost) from imported positions.

1. Press *CTRL-D* or click the Stats tab in the bottom panel.
2. Use the filter bar to restrict the analysis by player, tournament, date range, decision type, or match length.
3. Click an indicator to jump directly to the corresponding positions.

2.3.17 Calculate the EPC

The EPC (Effective Pip Count) calculator allows you to compute the bearoff statistics of a position.

1. Press *CTRL-E*, click the Bearoff tab in the bottom panel, or execute the `epc` command.
2. Edit the checker positions in the home board (last 6 points).
3. Results are displayed in real time in the dedicated Bearoff panel: EPC, average number of rolls, standard deviation, pip count and wastage — and, in pure bearoff, the winning probability of the player on roll with, within the exact domain, the money cube verdict (see the manual section "Methodology and assumptions of the Bearoff panel").
4. To practise estimating these values, tick the *Challenge* box: results are hidden on every change and are revealed area by area, with a click.

Note

The calculator works for both players simultaneously.

2.3.18 Display the analysis of a position imported from XG

If a position analyzed by XG, GNUbg, or BGBlitz has been imported into blunderDB, the analysis can be displayed by pressing *CTRL-L*.

If the position corresponds to a checker decision, the five best moves are displayed on separate lines. For each line, the information provided is in this order: the associated checker move, the normalized equity, the error in equity of the move, the winning chances, gammon, and backgammon for the player, and the winning chances, gammon, and backgammon for the opponent, along with the level of analysis.

If the position corresponds to a cube decision, the cost of each decision is displayed along with the winning chances of the position.

When multiple analysis engines are present for the same position (for example XG and GNUbg), an additional column indicates the source engine of each analysis.

When navigating a match, the actually played move is highlighted in the move list. If the position was encountered in multiple matches, all played moves are indicated.

Tip

By clicking on a move in the analysis panel, the corresponding arrows are displayed on the board.

2.3.19 Export a position to XG

To export a position from blunderDB to XG,

1. display the position to export in blunderDB and press *CTRL-C*,
2. open XG and press *CTRL-V*.

2.3.20 View the different positions

To view the different positions in the current library, use the *LEFT* and *RIGHT* keys. The *HOME* key allows you to go to the first position, and the *END* key lets you go to the last position.

To display the bearoff on the left, press *CTRL-LEFT*. To display the bearoff on the right, press *CTRL-RIGHT*.

2.3.21 Search for positions based on criteria

To search for types of positions,

- press *TAB* to open the search panel,
- edit the structure of the position to search for. blunderDB will filter positions that have at least the entered checker structure. If unsure, to maximize results, clear the position by pressing the *BACKSPACE* key. Edit the cube position and the score if necessary.

The search panel offers two checker structures, selected with the *At least* and *Except* tabs at the top of the panel:

- *At least* (default): blunderDB filters positions that have at least the entered checker structure;
- *Except*: blunderDB excludes positions that contain one of the entered checkers. The board is outlined in red while editing this structure. A position is rejected if it contains *at least one* of the drawn elements (for example, drawing a checker on points 1, 3 and 5 keeps only positions with no checker on any of these points). The number of checkers per point is not limited: setting 3 checkers on a point excludes positions with 3 or more checkers there (useful to search for a made point without a spare). Two quick clicks on a point mark it as required to be empty (a red hatched cell, no checker of any colour); a single click on that point unblocks it.

When a point belongs to both structures, the *Except* criterion prevails if it contradicts the *At least* criterion.

Method 1 (simple):

- Open the search window (*CTRL-F*)
- Add and set the search filters
- Confirm by clicking on “Search”.

Method 2 (advanced):

- open the command line by pressing *SPACE*,
- type *s*, and add any additional filters (for example, *cube* or *score* to consider the cube and score, respectively. See Section 2.4.4 for a comprehensive list of available filters).
- confirm the request by pressing *ENTER*.

The displayed positions are those from the database that meet the search criteria entered by the user.

2.4 List of commands

The command line, located in the status bar, opens by pressing the *SPACE* key. As you type a command, a list of suggestions appears automatically: the *TAB* key (or *SHIFT-TAB*) cycles through the suggestions and completes the command, while *ESC* closes the list (a second *ESC* closes the command line). The *UP* and *DOWN* keys remain reserved for the command history.

2.4.1 Global operations

Command	Action
new, ne, n	Create a new database.
open, op, o	Open an existing database.
import_db, idb	Import and merge another database.
export_db, edb	Export the current selection to a new database.
quit, q	Close blunderDB.
help, he, h	Open blunderDB help.
tutorial, tour	Opens the catalogue of guided interface tours.
demo	Loads a sample database (matches, tournament, analyses) to explore the tool.
meta	Display database metadata.
epc	Opens the Bearoff panel (Effective Pip Count, winning probability and cube verdict in bearoff).
met	Open the Kazaross-XG2 match equity table.
tp2	Open the takepoint table with a 2-cube.
tp2_live	Open the takepoint table with a 2-cube for long race positions.
tp2_last	Open the takepoint table with a 2-cube for last roll positions.
tp4	Open the takepoint table with a 4-cube.
tp4_live	Open the takepoint table with a 4-cube for long race positions.
tp4_last	Open the takepoint table with a 4-cube for last roll positions.
gv1	Open the gammon value table with a 1-cube.
gv2	Open the gammon value table with a 2-cube.
gv4	Open the gammon value table with a 4-cube.

2.4.2 Positions and navigation

Command	Action
import, i	Import one or more positions/matches from file (xg, xgp, sgf, mat, txt, bgf).
delete, del, d	Delete the current position.
[number]	Go to the specified index position.
list, l	Show the analysis of the current position.
comment, co	Show/write comments.
history, hi	Open the search panel (the search history is in its *History* tab).
stats, st	Show/hide the statistics panel.
match, ma	Show/hide the match panel.
collection, coll	Show/hide the collections panel.
#tag1 tag2 …	Tag the current position.
e	Load all positions from the database.
blunders, bl [n]	Load the worst mistakes (equity/MWC) into the analysis view, according to the current statistics filter. An optional number chooses how many to load (`bl 50`); 10 by default.Load the worst mistakes (equity/MWC) into the analysis view, according to the current statistics filter.
m	Navigate to the last visited match.

2.4.3 Editing and search

Command	Action
write, wr, w	Save the current position.
write!, wr!, w!	Update the current position.
s	Search for positions with filters.
ss	Search among the currently filtered positions.

2.4.4 Search Filters

The filters below must be juxtaposed during a search, i.e., after the start of the `s` command.

Warning

In the position search, by default, blunderDB takes into account the current checker structure, ignoring the position of the cube, the score, and the dice. To consider the position of the cube, the score, and the dice, it must be explicitly mentioned in the search.

Note

The search command `s` is available in the search panel (`TAB` key). The `ss` command allows searching among the currently filtered results.

Note

blunderDB considers that a backchecker is a checker located between point 24 and point 19.

Note

blunderDB considers that the number of checkers in the zone is the number of checkers located between point 12 and point 1.

Note

blunderDB considers the outfield to extend between point 18 and point 7.

Note

blunderDB considers the jan to extend between point 1 and point 6.

Tip

The parameters for filtering positions can be combined arbitrarily.

Query	Action
cube, cub, cu, c	The position checks the cube configuration.
score, sco, sc, s	The position checks the score.
d	The position checks the dice or the cube decision.
D	The position matches the dice roll (both dice, any order).
D1	The position matches the dice roll on the first die only (the first die's value appears on either die of the position).
xD65	The position was **not** played with the 6-5 roll (any order). The value is given in the token; repeatable to exclude several rolls (`xD65 xD54`).
nc	The position has no contact.
M	The position or the mirror one meets the filters.
i	The position was imported on its own, not brought in by a match import.
fl	The position was flagged in the source software, when importing an eXtreme Gammon match.
x	The position contains none of the checkers of the exclusion structure (the "Except" tab of the search panel).
p>x	The player has at least x pips behind in the race.
p<x	The player has at most x pips behind in the race.
px,y	The player has between x and y pips behind in the race.
P>x	The player has a race of at least x pips.
P<x	The player has a race of at most x pips.
Px,y	The player has a race between x and y pips.
e>x	The equity (in millipoints) of the position is greater than x.
e<x	The equity (in millipoints) of the position is less than x.
ex,y	The equity (in millipoints) of the position is between x and y.
E>x	The error of the move played by player 1 (in millipoints) is greater than x.
E<x	The error of the move played by player 1 (in millipoints) is less than x.
Ex,y	The error of the move played by player 1 (in millipoints) is between x and y.
w>x	The player has winning chances greater than x%.
w<x	The player has winning chances less than x%.
wx,y	The player has winning chances between x% and y%.
g>x	The player has gammon chances greater than x%.
g<x	The player has gammon chances less than x%.
gx,y	The player has gammon chances between x% and y%.
b>x	The player has backgammon chances greater than x%.
b<x	The player has backgammon chances less than x%.
bx,y	The player has backgammon chances between x% and y%.
W>x	The opponent has winning chances greater than x%.
W<x	The opponent has winning chances less than x%.
Wx,y	The opponent has winning chances between x% and y%.
G>x	The opponent has gammon chances greater than x%.
G<x	The opponent has gammon chances less than x%.
Gx,y	The opponent has gammon chances between x% and y%.
B>x	The opponent has backgammon chances greater than x%.
B<x	The opponent has backgammon chances less than x%.
Bx,y	The opponent has backgammon chances between x% and y%.
o>x	The player has at least x checkers off.

continues on next page

Table 1 – continued from previous page

Query	Action
o<x	The player has at most x checkers off.
ox,y	The player has between x and y checkers off.
O>x	The opponent has at least x checkers off.
O<x	The opponent has at most x checkers off.
Ox,y	The opponent has between x and y checkers off.
k>x	The player has at least x backcheckers.
k<x	The player has at most x backcheckers.
kx,y	The player has between x and y backcheckers.
K>x	The opponent has at least x backcheckers.
K<x	The opponent has at most x backcheckers.
Kx,y	The opponent has between x and y backcheckers.
z>x	The player has at least x checkers in the zone.
z<x	The player has at most x checkers in the zone.
zx,y	The player has between x and y checkers in the zone.
Z>x	The opponent has at least x checkers in the zone.
Z<x	The opponent has at most x checkers in the zone.
Zx,y	The opponent has between x and y checkers in the zone.
bo>x	The player has at least x blots in the outfield.
bo<x	The player has at most x blots in the outfield.
box,y	The player has between x and y blots in the outfield.
BO>x	The opponent has at least x blots in the outfield.
BO<x	The opponent has at most x blots in the outfield.
BOx,y	The opponent has between x and y blots in the outfield.
bj>x	The player has at least x blots in the jan.
bj<x	The player has at most x blots in the jan.
bjx,y	The player has between x and y blots in the jan.
BJ>x	The opponent has at least x blots in the jan.
BJ<x	The opponent has at most x blots in the jan.
BJx,y	The opponent has between x and y blots in the jan.
t'word1;word2;…'	The position comments contain at least one of the words.
co	The position carries a comment, whatever its content.
xco	The position carries no comment.
m'pattern1,pattern2,…'	The best checker moves containing at least one of the patterns.
m'ND,DT,DP,…'	The best cube decisions for No Double/Take, Double Take, Double Pass.
T>x	Date of position addition after x (YYYY/MM/DD).
T<x	Date of position addition before x (YYYY/MM/DD).
Tx,y	Date of position addition between x and y (YYYY/MM/DD).
max	Search in match with ID x (e.g. ma3).
max,y	Search in matches with IDs from x to y (e.g. ma2,5).
tnx	Search in tournament with ID x (e.g. tn1).
tnx,y	Search in tournaments with IDs from x to y (e.g. tn1,3).
idx	Search for the position with identifier x (e.g. id12).
idx,y	Search for the positions with identifiers x to y (e.g. id5,10).
pl'name'	Search positions from a match involving the named player, at either seat (e.g. pl'Alice'). Case-insensitive.

Note

Filtering positions based on the dice roll (*D* or *D1*) necessarily implies filtering positions based on the type of decision (*d*). The *D1* filter ignores the second die: only the first die's value is used to match positions (against either die of the

roll).

Note

For the relative difference filter in the race (*p>x*, *p<x*, *px,y*), the player is behind in the race compared to the opponent if *x>0* and ahead if *x<0*. For example: *p<-10*: the player is at least 10 pips ahead in the race. *p50,70*: the player is between 50 and 70 pips behind in the race.

Note

To search across several non-contiguous matches, repeat the `ma` filter (e.g. `s ma23 ma43` for matches 23 and 43). The same principle applies to tournaments with `tn` and to positions with `id` (e.g. `s id5 id10` for positions 5 and 10).

Note

Searching in a tournament (`tn`) means searching in all the matches of that tournament. Match and tournament IDs are visible in the ID columns of the corresponding panels.

For example, the command `s s c p-20,-5 w>60 z>10 K2,3` filters all positions taking into account the checker structure, the score, and the cube of the edited position where the player has between 20 and 5 pips ahead in the race, with at least 60% winning chances, at least 10 checkers in the zone, and the opponent has between 2 and 3 backcheckers.

2.4.5 Various commands

Command	Action
clear, cl	Clear the command history.

Note

Database migrations are now performed automatically when opening a database.

2.5 Keyboard shortcuts

Note

Keyboard shortcuts are independent of the keyboard layout: they remain accessible the same way regardless of the layout used (AZERTY, QWERTY, QWERTZ, etc.).

Note

When the cursor is inside a text field (comment, search box, command line), the usual text-editing shortcuts apply to the text rather than to the position: CTRL-C, CTRL-X and CTRL-V copy, cut and paste the selection, CTRL-A selects all of it, CTRL-Z and CTRL-Y undo and redo.

2.5.1 Database

Shortcut	Action
CTRL-N	Create a new database.
CTRL-O	Open an existing database.
CTRL-SHIFT-I	Import a database.
CTRL-SHIFT-S	Export the database.
CTRL-Q	Close blunderDB.
CTRL-M	Edit database metadata.

2.5.2 Position

Shortcut	Action
CTRL-I	Import one or more positions/matches from file (xg, xgp, sgf, mat, txt, bgf).
CTRL-SHIFT-F	Recursively import a folder of match/position files.
CTRL-C	Copy a position to the clipboard.
CTRL-X	Copy board image to clipboard (PNG).
CTRL-X CTRL-X	Copy board + analysis image to clipboard (PNG).
CTRL-V	Paste a position from the clipboard (automatic format detection).
CTRL-S	Save a position.
CTRL-U	Update a position.
Del	Delete the current position.
BACKSPACE	Reset board, cube, score, and dice.
CTRL-G	Show the position metadata.

2.5.3 Navigation

Shortcut	Action
CTRL-R	Reload all the positions from the database.
PageUp, h	First position / Previous game (match navigation).
LEFT, k	Previous position.
RIGHT, j	Next position.
UP, k	Previous move (when a move is selected in the analysis).
DOWN, j	Next move (when a move is selected in the analysis).
PageDown, l	Last position / Next game (match navigation).
r	Load a random position.

2.5.4 Display

Shortcut	Action
CTRL-LEFT	Board orientation to the left.
CTRL-RIGHT	Board orientation to the right.
p	Show/hide pipcount.

2.5.5 Actions

Shortcut	Action
TAB	Open the search panel (position editor).
SPACE	Open the command line.

2.5.6 Tools

Shortcut	Action
CTRL-L	Show/hide the analysis.
CTRL-P	Show/hide the comments.
CTRL-K	Show/hide the Anki panel (spaced repetition).
CTRL-F	Show/hide the search panel.
CTRL-Tab	Show/hide the match panel.
CTRL-B	Show/hide the collections panel.
CTRL-Y	Show/hide the tournaments panel.
CTRL-D	Show the Stats panel.
CTRL-E	Show/hide the Bearoff panel.
?	Show/hide the help.

2.5.7 View Tabs

Shortcut	Action
CTRL-T	Create a new view (a copy of the current view).
CTRL-W	Close the current view.
CTRL-PageUp, SHIFT-J	Previous view.
CTRL-PageDown, SHIFT-K	Next view.
CTRL-1 … CTRL-9	Go directly to the n-th view.
Double-click the tab	Rename the view.

2.5.8 Command Line

Shortcut	Action
UP	Browse command history up.
DOWN	Browse command history down.

2.5.9 Search History

The search history is the *History* tab of the search panel (*CTRL-F* or *TAB*).

Shortcut	Action
Click	Select/deselect a search (show position).
Double-click	Execute the search.

2.5.10 Filter Library

The filter library is the *Saved* tab of the search panel (*CTRL-F* or *TAB*).

Shortcut	Action
Click	Select/deselect a filter (show position).
Double-click	Execute the filter search.

2.5.11 Analysis Panel

Shortcut	Action
Click	Select/deselect a move (show/hide arrows).
UP, k	Select previous move (when a move is selected).
DOWN, j	Select next move (when a move is selected).
d	Toggle between checker and cube analysis (match navigation only).
Esc	Deselect move. If no move selected, close the panel.

2.5.12 Match Panel

Shortcut	Action
Click	Select a match.
Double-click	Navigate the match.
UP, k	Select previous match.
DOWN, j	Select next match.
ENTER	Load the selected match.
Del	Delete the selected match.
Esc	Deselect/close the panel.

2.5.13 Anki panel (spaced repetition)

Shortcut	Action
1	Rate: Again (failed, review soon).
2	Rate: Hard.
3	Rate: Good.
4	Rate: Easy.
Esc	Stop the review and return to the deck list (can resume later).

2.6 Command Line Interface (CLI)

2.6.1 Introduction

blunderDB ships a full command line interface (CLI) in the same executable as the graphical interface. The CLI is especially useful for:

- **bulk import** of matches: import an entire directory of match files (XG, SGF, MAT, BGF…) in a single command,
- **automation**: integrate blunderDB into shell scripts for regular backups, scheduled exports, or processing pipelines,
- **server usage**: manage databases on machines without a graphical environment,
- **quick inspection**: check the contents or integrity of a database without launching the graphical interface.

The CLI shares exactly the same database format as the graphical interface. Any operation performed via the CLI is immediately visible in the GUI and vice versa.

2.6.2 General syntax

The mode is detected automatically: if the first argument is a CLI command, blunderDB launches in headless mode, otherwise it launches the graphical interface.

```
# Mode graphique (aucun argument)
./blunderdb

# Mode CLI
./blunderdb <commande> [options]
```

2.6.3 Available commands

Command	Description
create	Create a new database.
import	Import data (match, position, batch).
export	Export data.
search	Search positions with filters.
list	Display database contents.
match	Display match positions and analysis.
info	Display database metadata.
edit	Edit database metadata.
verify	Verify database integrity.
delete	Delete data.
help	Show help.
version	Show version.

Each command accepts the `--help` option to display its detailed help.

2.6.4 create — Create a database

Create a new database file with optional metadata.

```
./blunderdb create --db <chemin> [--user <nom>] [--description <texte>] [--force]
```

Options:

- `--db` — Path to the database file to create (required).
- `--user` — Database owner name.
- `--description` — Database description.
- `--force` — Overwrite the file if it already exists.

The `.db` extension is added automatically if missing. Parent directories are created as needed.

Example:

```
./blunderdb create --db mes_matchs.db --user "Jean" --description "Matchs de tournoi
↪2025"
```

2.6.5 import — Import data

Import match or position files into the database.

```
./blunderdb import --db <chemin> --type <type> [options]
```

Options:

- `--db` — Path to the database (required).
- `--type` — Import type: `match`, `position` or `batch` (required).
- `--file` — File to import (for `match` and `position`).
- `--dir` — Directory to import (for `batch`).
- `--recursive` — Recursively scan subdirectories (default: yes).

Import a match

Supported formats: eXtreme Gammon (`.xg`, `.xgp`), GNUbg (`.sgf`), Jellyfish (`.mat`, `.txt`) and BGBlitz (`.bgf`).

```
./blunderdb import --db base.db --type match --file match.xg
```

Import positions

Import positions from a text file (one JSON position per line):

```
./blunderdb import --db base.db --type position --file positions.txt
```

Batch import

Import all match files from a directory in a single operation. This is the most efficient method for importing a large number of matches.

```
# Import récursif (par défaut)
./blunderdb import --db base.db --type batch --dir ./matchs/

# Import non récursif
./blunderdb import --db base.db --type batch --dir ./matchs/ --recursive=false
```

A summary table shows for each file whether the import succeeded (✓), failed (✗) or was a duplicate (⊘).

2.6.6 export — Export data

Export database contents to files.

```
./blunderdb export --db <chemin> --type <type> --file <sortie> [options]
```

Options:

- `--db` — Source database (required).
- `--type` — Export type: `database`, `positions`, `matches` or `mat` (export of one or more matches as a Jellyfish `.mat` transcript) (required).
- `--file` — Output file (required, except for `--type mat` used with `--dir`).
- `--dir` — Output directory for batch `.mat` export (several matches, one file per match; without `--match-ids`, all matches are exported).
- `--analysis` — Include analysis (default: yes).
- `--comments` — Include comments (default: yes).
- `--filters` — Include filter library (default: yes).
- `--played-moves` — Include played moves (default: yes).
- `--matches` — Include matches (default: yes).
- `--collections` — Include collections (default: no).
- `--collection-ids` — Collection IDs to export (comma-separated).
- `--match-ids` — Match IDs to export (comma-separated, empty = all).
- `--tournament-ids` — Tournament IDs to export (comma-separated).

Examples:

```
# Export complet de la base
./blunderdb export --db base.db --type database --file sauvegarde.db

# Export des positions en JSON
./blunderdb export --db base.db --type positions --file positions.txt

# Export de matchs spécifiques
./blunderdb export --db base.db --type matches --file selection.db --match-ids 1,3,5

# Export d'un match en transcription .mat (Jellyfish)
./blunderdb export --db base.db --type mat --match-ids 5 --file match5.mat

# Export de plusieurs matchs (ou de tous) en .mat dans un répertoire
./blunderdb export --db base.db --type mat --match-ids 5,9,12 --dir sorties/
./blunderdb export --db base.db --type mat --dir sorties/
```

2.6.7 search — Search positions

Search positions in the database using combinable criteria.

```
./blunderdb search --db <chemin> [options]
```

Main options:

- `--db` — Database (required).
- `--format` — Output format: `table`, `json` or `xgid` (default: `table`).
- `--limit` — Maximum number of results (0 = unlimited).
- `--export` — Export results to a new database.

Available filters:

- `--decision` — Decision type: `checker` or `cube`.
- `--dice` — Dice roll. `5,3` matches positions where both dice match (any order). `5` matches positions where a 5 appears on either die (the second die value is ignored). Implies `--decision checker` when no `--decision` value is given.
- `--pip-min` / `--pip-max` — Pip count difference range.
- `--winrate-min` / `--winrate-max` — Win rate range (%).
- `--cube` — Cube value.
- `--score1` / `--score2` — Player scores.
- `--match-length` — Match length.
- `--error-min` — Minimum equity error.
- `--move-error-min` / `--move-error-max` — Played move error (millipoints).
- `--has-analysis` — Only positions with analysis.
- `--off1-min` / `--off2-min` — Minimum checkers off (player 1/2).
- `--match-ids` — Filter by match IDs (comma-separated).
- `--tournament-ids` — Filter by tournament IDs (comma-separated).
- `--position-ids` — Filter by position IDs: range `2,7` (positions 2 to 7) or explicit semicolon-separated list `5;10;15`.
- `--individual` — Only positions imported on their own, that is, the ones you added yourself and not the ones a match import brought in.

Examples:

```
# Rechercher les décisions de videau
./blunderdb search --db base.db --decision cube

# Retrouver les positions que vous avez ajoutées vous-même
./blunderdb search --db base.db --individual

# Rechercher les positions avec erreur >= 0.1
./blunderdb search --db base.db --error-min 0.1

# Rechercher dans un tournoi et exporter
./blunderdb search --db base.db --tournament-ids 1 --export cubes.db

# Rechercher les positions avec un lancer de dés 6-5 (peu importe l'ordre)
./blunderdb search --db base.db --dice 6,5

# Rechercher les positions où un 6 a été obtenu sur l'un des deux dés
./blunderdb search --db base.db --dice 6
```

(continues on next page)

(continued from previous page)

```
# Sortie JSON limitée à 10 résultats
./blunderdb search --db base.db --format json --limit 10
```

2.6.8 list — List contents

Display database contents.

```
./blunderdb list --db <chemin> --type <type> [--limit <n>]
```

Types:

- `matches` — List of imported matches.
- `tournaments` — List of tournaments.
- `positions` — List of positions (limited to 10 by default).
- `stats` — Performance statistics report: PR / Snowie ER / MWC (overall, checker, cube), rolling PR over the last N decisions, top blunders, breakdown by cube action and error magnitude histogram.

Options (``stats`` type only):

- `--metric` — Displayed metric: `pr` or `mwc` (default: `pr`).
- `--player` — Restrict to the given player.
- `--tournament` — Restrict to one or more tournament IDs (comma-separated).
- `--from` — Start date (YYYY-MM-DD).
- `--to` — End date (YYYY-MM-DD).
- `--decision-type` — Decision type: `all`, `checker` or `cube` (default: `all`).
- `--top-blunders` — Number of worst errors listed (default: 10).
- `--format` — Output format: `text` or `json` (default: `text`).

Examples:

```
# Statistiques de la base
./blunderdb list --db base.db --type stats

# Statistiques en MWC pour un joueur donné
./blunderdb list --db base.db --type stats --metric mwc --player "Alice"

# Coups de pions uniquement, depuis une date
./blunderdb list --db base.db --type stats --decision-type checker --from 2026-01-01

# Sortie JSON (pour un script)
./blunderdb list --db base.db --type stats --format json

# Liste des matchs
./blunderdb list --db base.db --type matches

# Premières 20 positions
./blunderdb list --db base.db --type positions --limit 20
```

2.6.9 match — Display a match

Display positions and analysis of an imported match.

```
./blunderdb match --db <chemin> --id <id_match> [--format <format>] [--output
↪<fichier>]
```

Options:

- `--db` — Database (required).
- `--id` — Match ID to display (required).
- `--format` — Output format: `json`, `text` or `summary` (default: `json`).
- `--output` — Output file (default: stdout).

Examples:

```
# Résumé d'un match
./blunderdb match --db base.db --id 1 --format summary

# Détails de chaque position
./blunderdb match --db base.db --id 1 --format text

# Export JSON vers un fichier
./blunderdb match --db base.db --id 1 --output match1.json
```

2.6.10 info — Database metadata

Display metadata and statistics of a database.

```
./blunderdb info --db <chemin> [--format <format>]
```

Options:

- `--db` — Database (required).
- `--format` — Output format: `text` or `json` (default: `text`).

Examples:

```
# Afficher les informations
./blunderdb info --db base.db

# Sortie JSON (pour un script)
./blunderdb info --db base.db --format json
```

2.6.11 edit — Edit metadata

Edit the user name or description of a database.

```
./blunderdb edit --db <chemin> [options]
```

Options:

- `--db` — Database (required).
- `--user` — New user name.

- `--description` — New description.
- `--clear-user` — Clear user name.
- `--clear-description` — Clear description.

At least one edit option is required.

Examples:

```
# Modifier l'utilisateur et la description
./blunderdb edit --db base.db --user "Marie" --description "Ma collection"

# Effacer la description
./blunderdb edit --db base.db --clear-description
```

2.6.12 verify — Verify integrity

Verify database integrity and optionally compare a match against its source file.

```
./blunderdb verify --db <chemin> [--match <id>] [--mat <fichier.mat>]
```

Options:

- `--db` — Database (required).
- `--match` — Match ID to verify.
- `--mat` — MAT file to compare against (used with `--match`).

Without `--match`, the command displays general database statistics. With `--match`, it verifies the match data and can compare it against the original source file.

Examples:

```
# Vérification globale
./blunderdb verify --db base.db

# Vérifier un match spécifique
./blunderdb verify --db base.db --match 1

# Comparer avec le fichier source
./blunderdb verify --db base.db --match 1 --mat original.mat
```

2.6.13 delete — Delete data

Delete a match and all associated data (games, moves, analyses).

```
./blunderdb delete --db <chemin> --type match --id <id> [--confirm]
```

Options:

- `--db` — Database (required).
- `--type` — Delete type: `match` (required).
- `--id` — ID of the item to delete (required).
- `--confirm` — Delete without asking for confirmation.

Examples:

```
# Supprimer avec confirmation interactive
./blunderdb delete --db base.db --type match --id 1

# Supprimer sans confirmation (pour scripts)
./blunderdb delete --db base.db --type match --id 1 --confirm
```

2.6.14 Workflow examples

Import a tournament directory

```
# Créer une base dédiée au tournoi
./blunderdb create --db tournoi_paris.db --user "Jean" --description "Open de Paris␣
↪2025"

# Importer tous les matchs du répertoire
./blunderdb import --db tournoi_paris.db --type batch --dir ./matchs_open_paris/

# Vérifier le résultat
./blunderdb list --db tournoi_paris.db --type stats
```

Regular backup

```
# Export complet pour sauvegarde
./blunderdb export --db production.db --type database --file sauvegarde-$(date␣
↪+%Y%m%d).db
```

Error analysis

```
# Extraire les blunders dans une base séparée
./blunderdb search --db production.db --error-min 0.1 --export blunders.db

# Extraire les erreurs de videau
./blunderdb search --db production.db --decision cube --error-min 0.05 --export␣
↪cube_errors.db
```

2.6.15 Exit codes

- `0` — Success.
- `1` — Error.

This makes the CLI suitable for use in scripts with error handling:

```
if ./blunderdb import --db base.db --type match --file match.xg; then
    echo "Import réussi"
else
    echo "Échec de l'import"
    exit 1
fi
```

2.7 Frequently Asked Questions (FAQ)

2.7.1 What is the purpose of blunderDB?

blunderDB allows users to create a personalized database of positions. Its strength lies in not presupposing any classification *a priori*. This gives users the freedom to query positions with great flexibility by combining various criteria (race, structure, cube, score, backcheckers, checkers in the zone, chances of winning/gammon/backgammon, etc.).

Another convenient use of blunderDB is the creation of reference position catalogs. With the ability to tag positions, users can organize all their reference positions in a structured way using a single file. I hope that blunderDB facilitates the sharing of positions between players.

2.7.2 What motivated the creation of blunderDB?

I used to store interesting positions or blunders in different folders. However, I encountered difficulties in retrieving positions based on criteria that weren't initially considered in my choice of thematic categories. For example, if the positions were sorted by type of game (race, holding game, blitz, backgame, etc.), how could I retrieve all positions at a certain score or at a given cube level? Additionally, some old positions tended to be forgotten. I wanted a tool that aggregates all my positions without presupposing thematic categories *a priori*, allowing me to ask questions of the database. This type of software is quite common in chess, like ChessBase.

2.7.3 How to save the state of the current database?

The database is updated immediately upon validation of the request. No explicit saving operation is necessary.

2.7.4 Should I create different databases for different categories of positions?

Unless there are clearly identified reasons, it is essential not to split positions into separate databases, as this could prevent you from linking them in future searches. The philosophy of blunderDB is not to assume position categories *a priori* but to allow the user to query them flexibly. When positions have been encountered under specific conditions or for particular reasons, it may be wise to store them in separate databases. For example, you could create distinct position databases for:

- reference positions,
- blunders from live tournaments,
- blunders from online play.

2.7.5 How to merge multiple databases?

If you have multiple blunderDB databases that you wish to merge, use the "Import Database" feature:

1. Open the main database (the one that will receive the imported positions)
2. Click on the "Import Database" button in the toolbar
3. Select the database to import
4. blunderDB will automatically merge the positions

During the merge, blunderDB avoids duplicates and intelligently merges analyses and comments. Identical positions will not be duplicated, but their analyses and comments will be combined.

Note

It is recommended to make a backup copy of your main database before importing another database.

2.7.6 Which match file formats are supported?

blunderDB supports the following match formats:

- **eXtreme Gammon (XG)**: *.xg* files, with complete move analysis, cube decisions, played moves, and multi-engine support. *.xgp* files for importing individual positions with analysis.
- **GNUbg**: *.sgf* files (Smart Game Format), with analysis.
- **Jellyfish**: *.mat* and *.txt* files.
- **BGBlitz**: *.bgf* files and text positions.

Import can be done via a single file, multiple selection, recursive folder, paste from clipboard, or drag and drop.

blunderDB automatically detects duplicates and prevents importing a match already present in the database.

2.7.7 What is a collection?

A collection is a custom grouping of positions. Unlike a filter search which is dynamic, a collection is a fixed set of positions manually chosen by the user. Collections allow, for example, grouping reference positions for a particular topic.

2.7.8 What is EPC?

EPC (Effective Pip Count) is a more accurate measure than the simple pip count for evaluating bearoff positions. The blunderDB EPC calculator uses the GNUbg 6-point bearoff database and computes in real time the EPC, average number of rolls, standard deviation, pip count, and wastage.

On pure bearoff positions, the panel also shows the winning probability of the player on roll and, when the position is covered by a two-sided database (built-in database up to 6 checkers per player, downloadable database up to 11), the exact money cube verdict. Outside that domain, the probability is estimated with its error margin and the verdict is deliberately not shown. See the manual section “Methodology and assumptions of the Bearoff panel” for the details of the assumptions.

2.7.9 Does blunderDB have a command-line interface?

Yes, blunderDB has a command-line interface (CLI) that allows performing without a graphical interface operations such as creating databases, importing matches, exporting, searching positions, displaying statistics, etc. Refer to the CLI documentation for more details.

2.7.10 Can I modify, copy, share blunderDB?

Yes, absolutely. blunderDB is licensed under the MIT license.

2.7.11 What data format does blunderDB use?

The database is a simple SQLite file. In the absence of blunderDB, it can thus be opened with any SQLite file editor.

2.7.12 What were the design principles of blunderDB?

I wanted an interface that stays approachable, but designed above all for advanced, sustained use, where one moves from position to position and from search to search. The command line, opened by pressing the *SPACE* bar, and the keyboard shortcuts serve that use, without being mandatory.

I also wanted blunderDB to be lightweight, self-contained, installation-free and available on different platforms, hence my choice of the Go language and the Svelte library. For serialising the database, the file format had to be cross-platform and suited to holding a database. The SQLite file format seemed the obvious fit.

I was also keen that a database should remain a single file, one that can be copied, backed up or sent to another player.

Finally, blunderDB is no longer limited to the desktop application: the same binary provides a command-line interface and an optional server mode, which can rely on PostgreSQL for multi-user deployments. The normal way to use it nevertheless remains the desktop application. See *Command Line Interface (CLI)* and *Headless mode (server)*.

2.7.13 What is the software architecture of blunderDB?

- The backend is coded in Go. It is responsible for all operations on the SQLite database that stores the positions.
- The frontend is coded in Svelte. It is responsible for rendering the graphical interface and the Backgammon board.
- The application is encapsulated with Wails, enabling the production of native desktop applications for Windows, Linux and macOS.
- The database is managed by SQLite.
- The optional server mode can rely on PostgreSQL instead of SQLite for multi-user deployments.

For more information, see the blunderDB GitHub repository.

2.7.14 On which platforms does blunderDB run?

blunderDB runs on Windows, Linux and Mac.

2.7.15 Where does the blunderDB icon come from?

The blunderDB icon is the "goggling" emoticon from the SMirC series.

2.8 Headless mode (server)

Note

This section describes an **advanced, optional mode** of blunderDB, intended for server deployments, multi-user setups and automation. **The normal and recommended use of blunderDB remains the desktop application** described in the previous chapters. If you use blunderDB on your own, on your computer, you do not need this mode: you can skip this chapter without losing any of the analysis features.

2.8.1 Overview

In addition to the desktop application and the command-line commands (see *Command Line Interface (CLI)*), the same `blunderdb` binary can run in **headless mode**: without a graphical interface, driven entirely from the command line or over the network. This mode covers three use cases:

- **the** `serve` **daemon** — exposes the blunderDB engine as an HTTP + JSON service, to run a shared database on a server and access it from several clients;
- the generic `call` dispatcher — calls any storage operation directly, locally, for scripting and testing;
- the `migrate` command — transfers a single-user SQLite database to a multi-user PostgreSQL backend.

These three use cases rely on a shared **storage layer** that can talk to two backends: **SQLite** (the usual `.db` file format of the desktop application) and **PostgreSQL** (for multi-user server deployments).

2.8.2 The `serve` daemon

`blunderdb serve` runs the engine as an HTTP service that responds in JSON. It lets you host a position database on one machine and access it from several clients.

```
# Servir une base SQLite locale sur le port 8080
blunderdb serve --db ma_base.db --addr :8080

# Servir un backend PostgreSQL
blunderdb serve --backend postgres \
    --dsn "postgres://user:pass@host:5432/blunderdb?sslmode=disable" \
    --addr :8080
```

Warning

The daemon performs no authentication. It trusts the `X-Tenant-ID` request header and **must** run behind a reverse proxy (nginx, Caddy…) responsible for authentication. **Never expose it directly to the public Internet.**

Options:

Option	Default	Meaning
`--db <path>`	–	SQLite file (shorthand for `--backend sqlite --dsn <path>`)
`--backend <type>`	`sqlite`	storage backend: `sqlite` or `postgres`
`--dsn <string>`	`$BLUNDERDB_DSN`	backend connection string
`--addr <host:port>`	`:8080`	listen address
`--log-level <level>`	`info`	log level: `debug|info|warn|error`
`--metrics`	`true`	exposes `/metrics` (Prometheus format)
`--cors-allow-origin <origin>`	–	enables CORS for this origin (disabled by default)
`--rate-limit-rps <n>`	`0`	requests per second per tenant limit (0 = disabled)
`--rate-limit-burst <n>`	`2×rps`	token-bucket size for request bursts
`--rls`	`false`	PostgreSQL: enables per-tenant Row-Level Security (defence in depth, opt-in)
`--bearoff-ts <file>`	–	optional two-sided bearoff database (`.bd`) widening the built-in TS-06-06 database for the race analysis of the EPC endpoint; the daemon never downloads a database itself — mount the file as a volume and point to it here

Most options can also be provided through environment variables (`BLUNDERDB_BACKEND`, `BLUNDERDB_DSN`, `BLUNDERDB_ADDR`, `BLUNDERDB_LOG_LEVEL`, `BLUNDERDB_RLS`, `BLUNDERDB_TS_PATH`).

Endpoints

The service exposes operational endpoints, always present:

- `GET /healthz` — liveness (the process is running);
- `GET /readyz` — readiness (storage responds);
- `GET /metrics` — Prometheus metrics (if `--metrics` is enabled).

The business surface follows the `POST /v1/<family>.<method>` scheme (for example `/v1/positions.save`, `/v1/matches.get`). The families cover positions, analyses, matches, comments, collections, tournaments, Anki cards, filters, sessions, search, metadata, statistics and import. Listing endpoints return an NDJSON stream (one JSON object per line). The server shuts down cleanly on `SIGINT` / `SIGTERM`.

Two methods in the `positions` family decode a position without storing it: `positions.fromXGID` rebuilds a position from an XGID string, and `positions.fromXGP` from a single-position `.xgp` file.

The `anki` family gains six methods that extend the spaced-repetition scheduler (FSRS): `anki.reviewLog` (a log of every review — rating and FSRS outcome — powering retention statistics and a faithful history), `anki.forecast` (a projection of the number of cards due over the coming days, including overdue cards), `anki.suspendCard` / `anki.buryCard` / `anki.removeCard` (remove a card from the review queue temporarily or permanently) and `anki.optimizeParams` (nudges a deck's target retention rate toward the pass rate observed on its reviews).

Deployment with Docker

The repository provides a `Dockerfile.serve` that builds a minimal container image of the daemon: only the `serve` binary is compiled (pure Go, with no graphical interface and no CGO, therefore statically linked), then placed into a *distroless* image.

```
# Construire l'image (depuis la racine du dépôt)
docker build -f Dockerfile.serve -t blunderdb-serve .

# Lancer le démon (le backend par défaut de l'image est postgres)
docker run --rm -p 8080:8080 \
    -e BLUNDERDB_DSN="postgres://user:pass@hôte:5432/blunderdb?sslmode=disable" \
    blunderdb-serve
```

The image listens on port 8080 and is configured through environment variables (`BLUNDERDB_BACKEND`, `BLUNDERDB_DSN`, `BLUNDERDB_ADDR`, `BLUNDERDB_RLS`).

Warning

Like the daemon itself, the container performs **no authentication**: it must be placed behind a reverse proxy responsible for authentication and never be exposed directly to the public Internet.

2.8.3 PostgreSQL backend and multi-user

For a shared deployment, blunderDB can store data in **PostgreSQL** rather than in a SQLite file. The backend is selected with `--backend postgres` and the `--dsn` connection string. The schema is created and migrated automatically at startup.

Data is **partitioned per tenant**: each request carries a scope identifier (the `X-Tenant-ID` header, `default` by default), which lets several users share the same instance without seeing each other's data. The `--rls` option additionally enables PostgreSQL's **Row-Level Security**: per-tenant isolation policies are installed and `app.tenant_id` is set per connection. This is an optional defence in depth, disabled by default.

When a tenant is decommissioned, `POST /v1/tenant.purge` permanently deletes all its data (positions, matches, collections, history, etc.) for the current tenant (the one carried by `X-Tenant-ID`), **as well as its session state** (last search, last position, open tabs — the handful of `metadata` rows prefixed with that scope): the operation runs in a single transaction, is idempotent (no error purging an already-empty tenant or repeating the call), and does not affect any other tenant or the global schema-version row. It is only available with the PostgreSQL backend — it returns an `invalid` error on a SQLite backend, which has no notion of tenant.

2.8.4 Migrating a SQLite database to PostgreSQL

`blunderdb migrate` copies a single-user SQLite database to a PostgreSQL backend, under a chosen tenant scope — this is the path to "upload" a desktop library to a server deployment.

```
blunderdb migrate \
    --from sqlite:///chemin/vers/base.db \
    --to   "postgres://user:pass@host:5432/db?sslmode=disable" \
    --tenant-id mon-tenant

# Prévisualiser sans rien écrire
blunderdb migrate --from sqlite:///chemin/vers/base.db \
    --tenant-id mon-tenant --dry-run
```

The migration copies the **positions, their analyses and comments, the matches (games + moves), the tournaments (with their match links) and the collections (with their composition)**, reassigning primary and foreign keys, all within a **single transaction** on the destination side: the operation is atomic (a failure leaves the destination intact, just run it again). Progress and the final summary are emitted as NDJSON on standard output.

Option	Default	Meaning
`--from <uri>`	–	source SQLite database (`sqlite:///path` or a plain path)
`--to <dsn>`	–	destination PostgreSQL DSN (`postgres://…`)
`--tenant-id <scope>`	–	destination tenant scope (required except in `--dry-run`)
`--dry-run`	–	counts what would be copied without writing anything
`--on-conflict <policy>`	`""`	`""` aborts if the tenant already has data; `skip` merges (deduplication of positions by Zobrist hash)

Note

Application state is not (yet) migrated: Anki decks/cards, the filter library, search and command history, and session metadata. The priority is migrating the position library and the match history.

2.8.5 The generic `call` dispatcher

In addition to the historical subcommands (*Command Line Interface (CLI)*), `blunderdb call` exposes **all** storage operations directly, locally. It goes through the same handlers as the `serve` daemon: the behaviour is therefore identical to `POST /v1/<family>.<method>`. This is useful for scripting and integration testing.

```
# Lister toutes les méthodes disponibles
blunderdb call --list

# Lectures
blunderdb call metadata.counts --db ma_base.db
blunderdb call positions.list  --db ma_base.db --json '{"limit":10}'
blunderdb call matches.get     --db ma_base.db --json '{"id":1}'

# Écritures
blunderdb call positions.save  --db ma_base.db --json '{"position":{...}}'
blunderdb call matches.delete  --db ma_base.db --json '{"id":42}'
```

Options:

Option	Default	Meaning
`--db <path>`	–	SQLite file (shorthand for `--backend sqlite --dsn <path>`)
`--backend <type>`	`sqlite`	`sqlite` or `postgres`
`--dsn <string>`	`$BLUNDERDB_DSN`	backend connection string
`--scope <string>`	`default`	tenant scope (sent as `X-Tenant-ID`)
`--json <string>`	`{}`	request body in JSON format
`--json-file <path>`	–	reads the request body from a file
`--list`	–	lists all `<family>.<method>` methods and exits

The JSON response (or the NDJSON stream for `*.list` endpoints) is written to standard output. On error, the process exits with a non-zero code and the `{"error":{…}}` envelope is printed to standard output so it stays parseable (for example with `jq`).

2.9 Annex: Advanced Filter Usage

Warning

This section is for advanced users of blunderDB who wish to fully leverage the position search features.

Filters are at the heart of position analysis in blunderDB. Their use allows for searching specific positions with great precision. In this section, the use of filters through the command line is detailed. The command line can be accessed by pressing the `SPACE` key. It allows users to quickly combine filters and use the filter library with ease.

2.9.1 Command-line search for positions

To perform a search using filters,

1. Press the `TAB` key to open the search panel.
2. Edit the current position.
3. Open the command line using the `SPACE` key.
4. Use the command `s` followed, optionally, by filters.
5. Start the search with the `ENTER` key.

Warning

Don't forget to clear the current position before starting a search (`BACKSPACE` key), if it isn't the one you want, to avoid excessively filtering checker structures.

Note

The list of available filters in the command line is provided in Section 2.4.4.

2.9.2 Search in Current Results

It is possible to refine a search by searching among the currently filtered positions. This allows you to progressively narrow down the results.

In the command line, use the `ss` command followed by filters (e.g.: `ss nc`, `ss E>40`). The `ss` command works after a prior search.

The search window (`CTRL-F`) also offers a "Search in current results" checkbox for the same functionality.

2.9.3 Filter Library

The filter library allows the user to save search commands to facilitate their thematic studies.

To add a filter to the library,

1. Start the search you want to keep (`s` command followed by filters).
2. Open the search panel (`TAB` or `CTRL-F`) and select the *History* tab.
3. Click the bookmark icon of the search to save.
4. Give the filter a name and confirm.

To use a filter saved in the library,

1. Open the search panel (`TAB` or `CTRL-F`) and select the *Saved* tab.
2. Search for the desired filter.
3. Double-click on the filter to start the search.

2.9.4 Examples

Here are some examples of using filters in the command line:

Position type	Checker structure	Command
Pure race		s nc
Short race		s nc P<70
Hitting on the ace point		s m"6/1*"
Backgame 1-4	points 24, 21 made	s p>35
Take/Pass decision at -2 -4	empty dice on upper player's side, score -2/-4	s s d
Too good to double	empty dice on lower player's side	s d e>1000
Blitz with at least 20% gammon	made points in home board, men on the bar	s g>20
Player 1 errors greater than 40 millipoints		s E>40
Position from the Aachen2024 tournament		s t"Aachen2024"
One back checker to bring home		s k1,1
Escaping from the 20 point	point 20	s m"20/"
Prime versus prime	indicate the primes	s
Ace-point bear-off	opponent's ace point made	s P<60
Double with at least 20 pip lead	empty dice on lower player's side	s d p<-20
Positions from match 5		s ma5
Positions from matches 2 to 4		s ma2,4
Positions from matches 23 and 43		s ma23 ma43
Positions from tournament 1		s tn1
Errors in tournament 2		s tn2 E>40

Examples of searching within current results:

Scenario	Command
After `s nc`, search for short races	ss P<70
After `s g>20`, keep only errors	ss E>40
After `s tn1`, search for cube decisions	ss d

2.10 Windows Annex: False Detection of blunderDB as Malware

Note

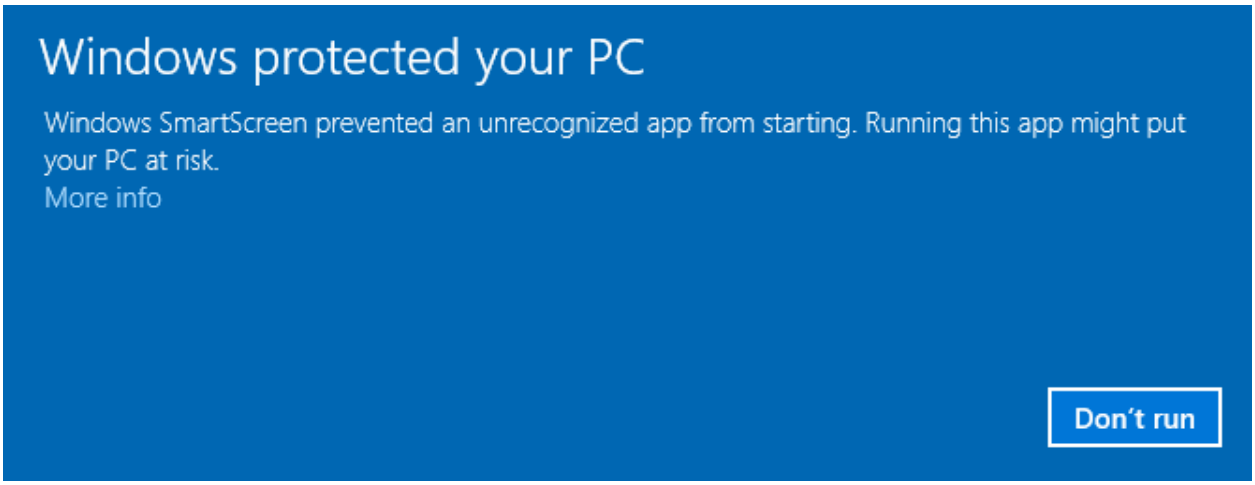
The following applies to Windows 10 and 11 operating systems.

Windows now requires software publishing companies or independent software developers to digitally certify their applications, or even distribute them via the Windows Store. It is therefore recommended to turn to external companies to obtain a digital certificate, which costs several hundred euros (see, for example, https://learn.microsoft.com/en-us/archive/blogs/ie_fr/certificats-de-signature-de-code-ev-extended-validation-et-microsoft-smartscreen).

Since I am offering blunderDB for free, I do not wish to pursue these costly options. A **free** avenue reserved for open-source software (the *SignPath Foundation*, https://signpath.org/) was explored, but the application was unsuccessful; the Windows binaries are therefore not digitally signed. Consequently, it is very likely that Windows will warn you of a potential threat or even block the execution of blunderDB entirely. The following sections explain the steps to bypass Windows' reluctance, and how to verify the integrity of the downloaded binary.

2.10.1 Windows SmartScreen Warning

After downloading blunderDB, when you run it, Windows may display a warning such as:

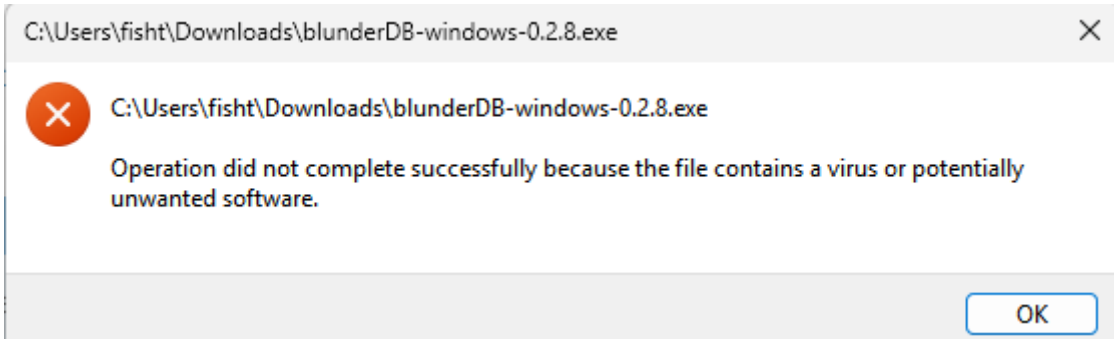


If you want to allow a specific executable blocked by SmartScreen:

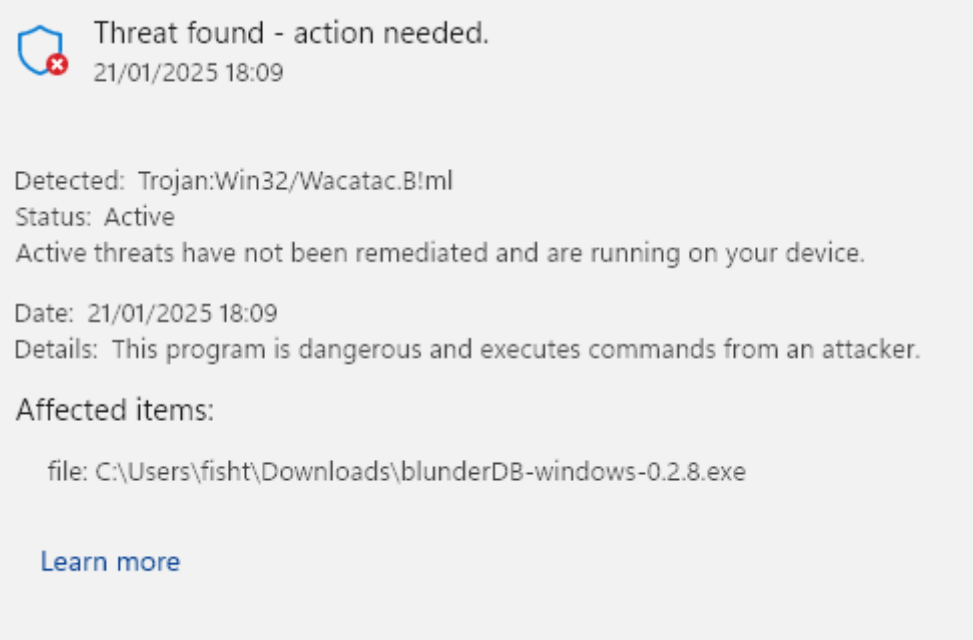
1. **Try running the executable**:
   - When you attempt to launch the executable, SmartScreen may block it and display a warning.
2. **Click on "More Info"**:
   - In the SmartScreen warning window, click on **More Info**.
3. **Select "Run anyway"**:
   - If you trust the executable, click **Run anyway** to bypass the SmartScreen warning for this instance.

2.10.2 Windows Defender Blocking

For certain security settings in Windows, even after bypassing SmartScreen (see the previous section), Windows Defender may prevent the execution of blunderDB with messages such as:



or even:



or even place it in quarantine.

Windows Defender is known to trigger false positives. This issue is explicitly mentioned in the FAQ on the official Golang website (https://go.dev/doc/faq#virus) or in GitHub tickets for some projects programmed in Go (https://github.com/golang/vscode-go/issues/3182).

If you want to prevent Windows Security from scanning blunderDB:

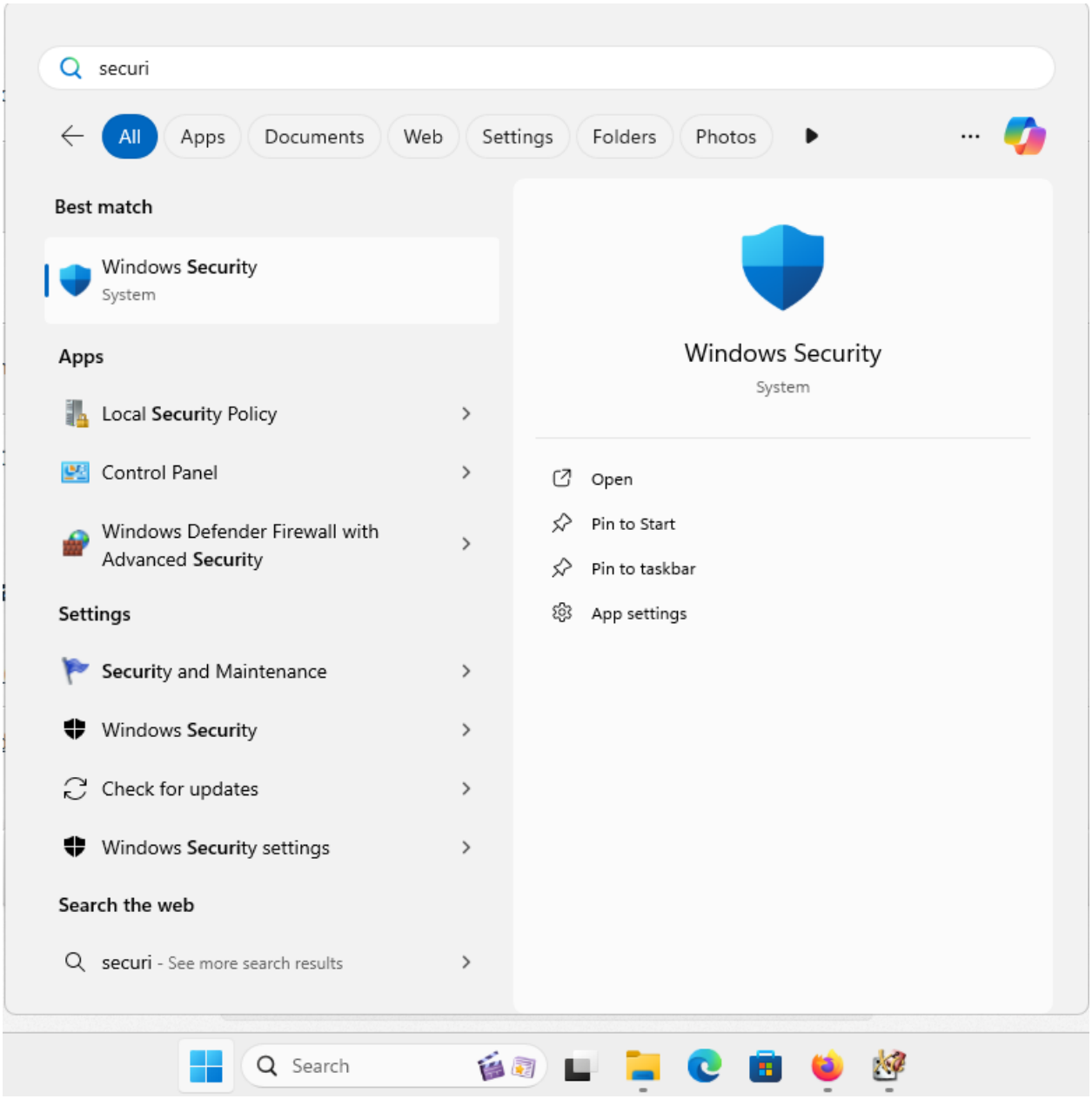
1. **Open Windows Security**:
   - Go to **Start** and type **Windows Security**.
2. **Go to "Virus & Threat Protection"**:
   - Click on **Virus & Threat Protection**.
3. **Manage Settings**:
   - Scroll down and click on **Manage settings** under Virus & Threat Protection settings.
4. **Add or remove exclusions**:
   - Scroll down to the **Exclusions** section and click on **Add or remove exclusions**.
5. **Add an exclusion**:
   - Click on **Add an exclusion** and select **File**. Then, navigate to the executable you want to exclude and select it.

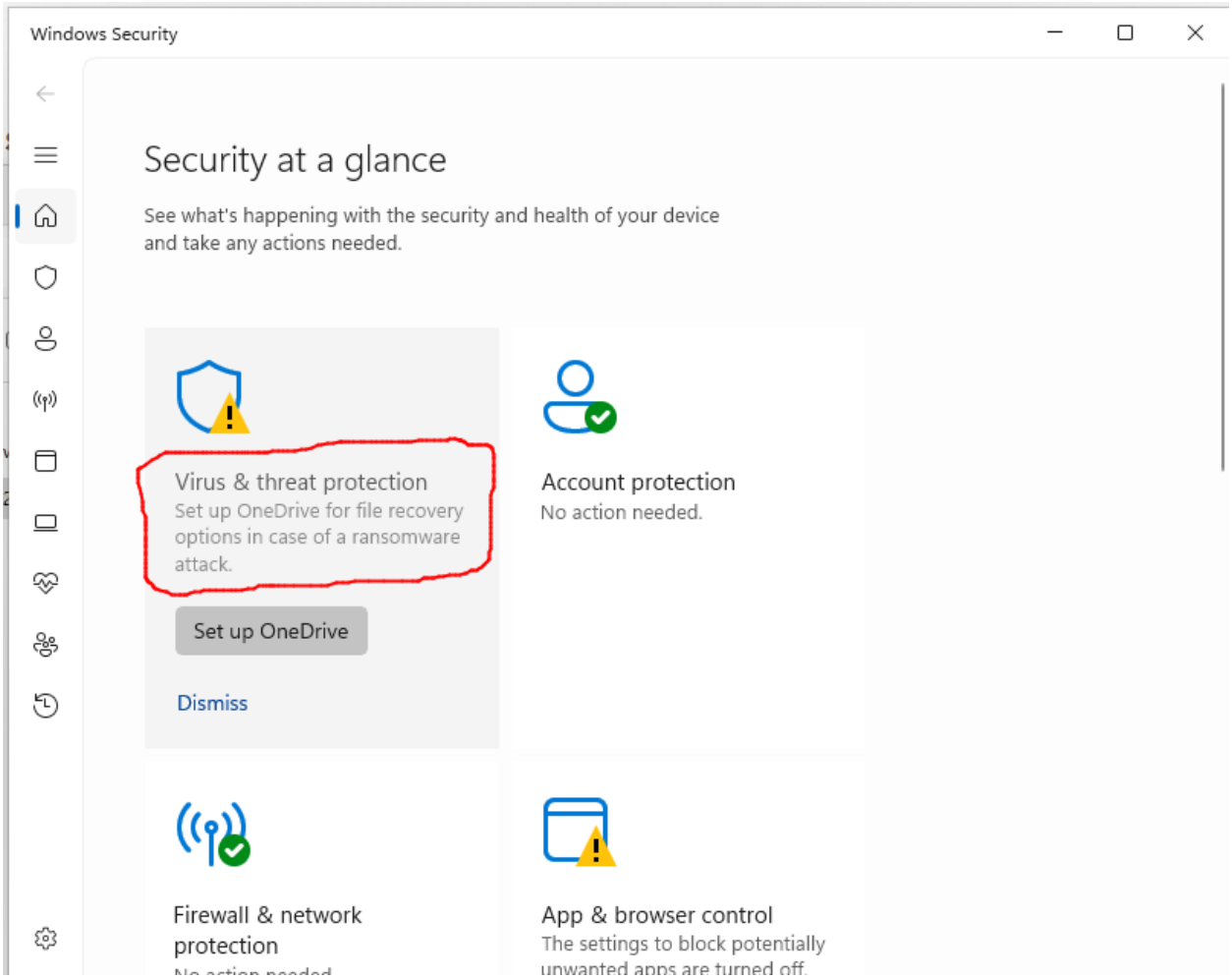
2.10.3 Verify the download integrity (SHA256)

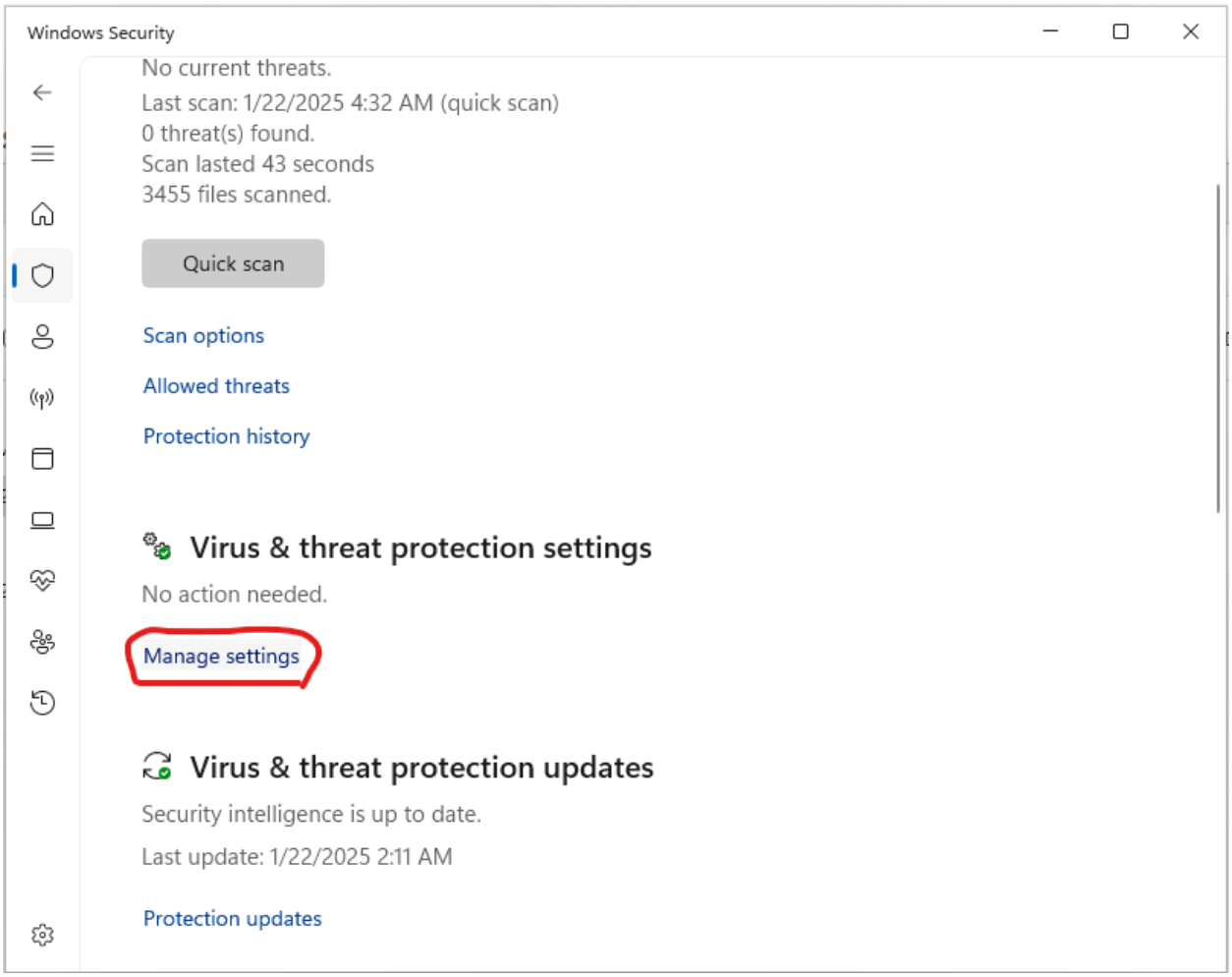
Each binary published on the *releases* page comes with a `.sha256` file containing its cryptographic fingerprint. Checking this fingerprint ensures that the downloaded file is authentic and has not been tampered with, which is a useful guarantee in the absence of code signing.

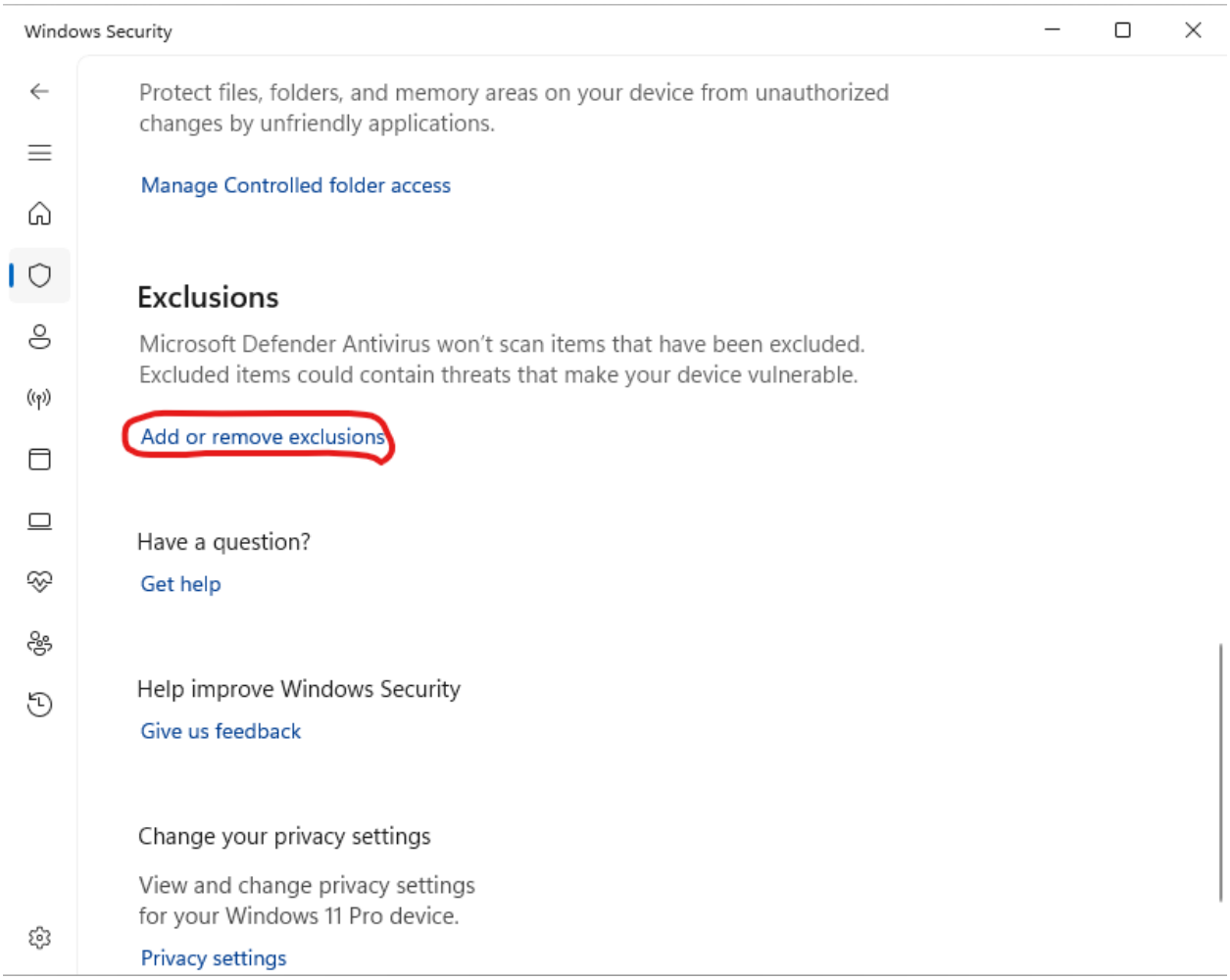
On Windows (PowerShell), in the download folder:

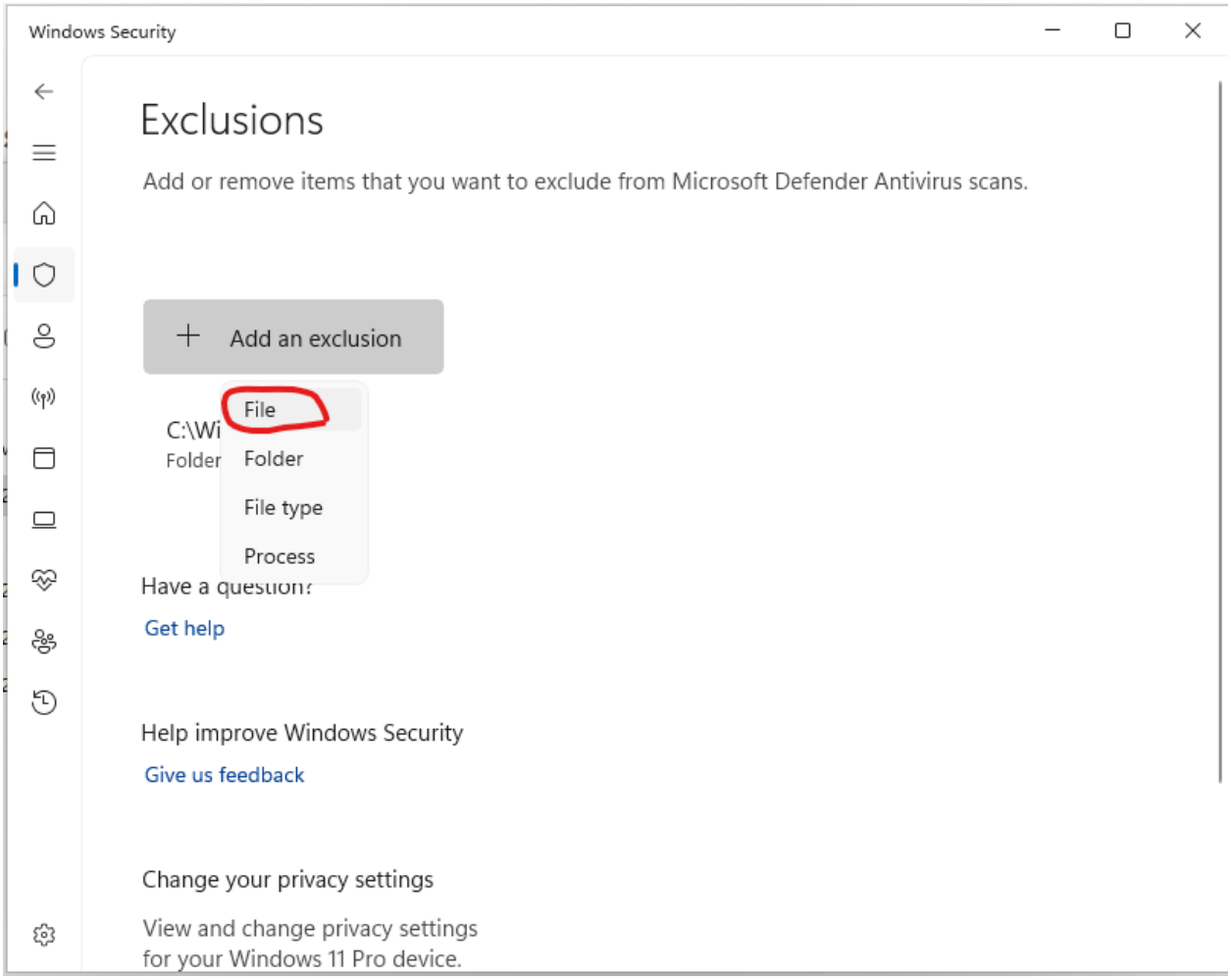
```
Get-FileHash .\blunderDB-windows-<version>.exe -Algorithm SHA256
```

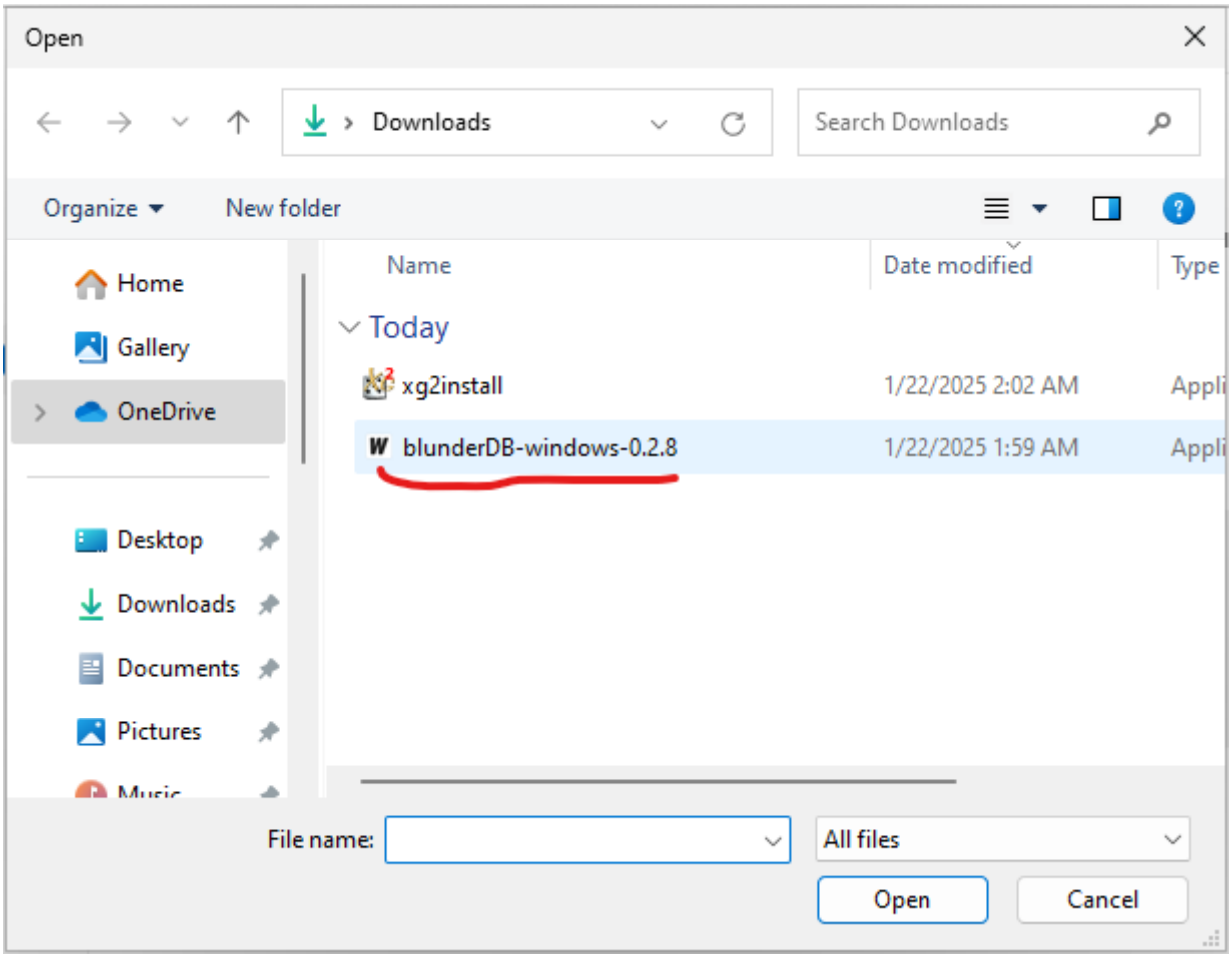

securi
All
Apps
Documents
Web
Settings
Folders
Photos
Best match
Windows Security
System
Apps
Local Security Policy
Control Panel
Windows Defender Firewall with Advanced Security
Settings
Security and Maintenance
Windows Security
Check for updates
Windows Security settings
Search the web
securi - See more search results
Windows Security
System
Open
Pin to Start
Pin to taskbar
App settings
Search

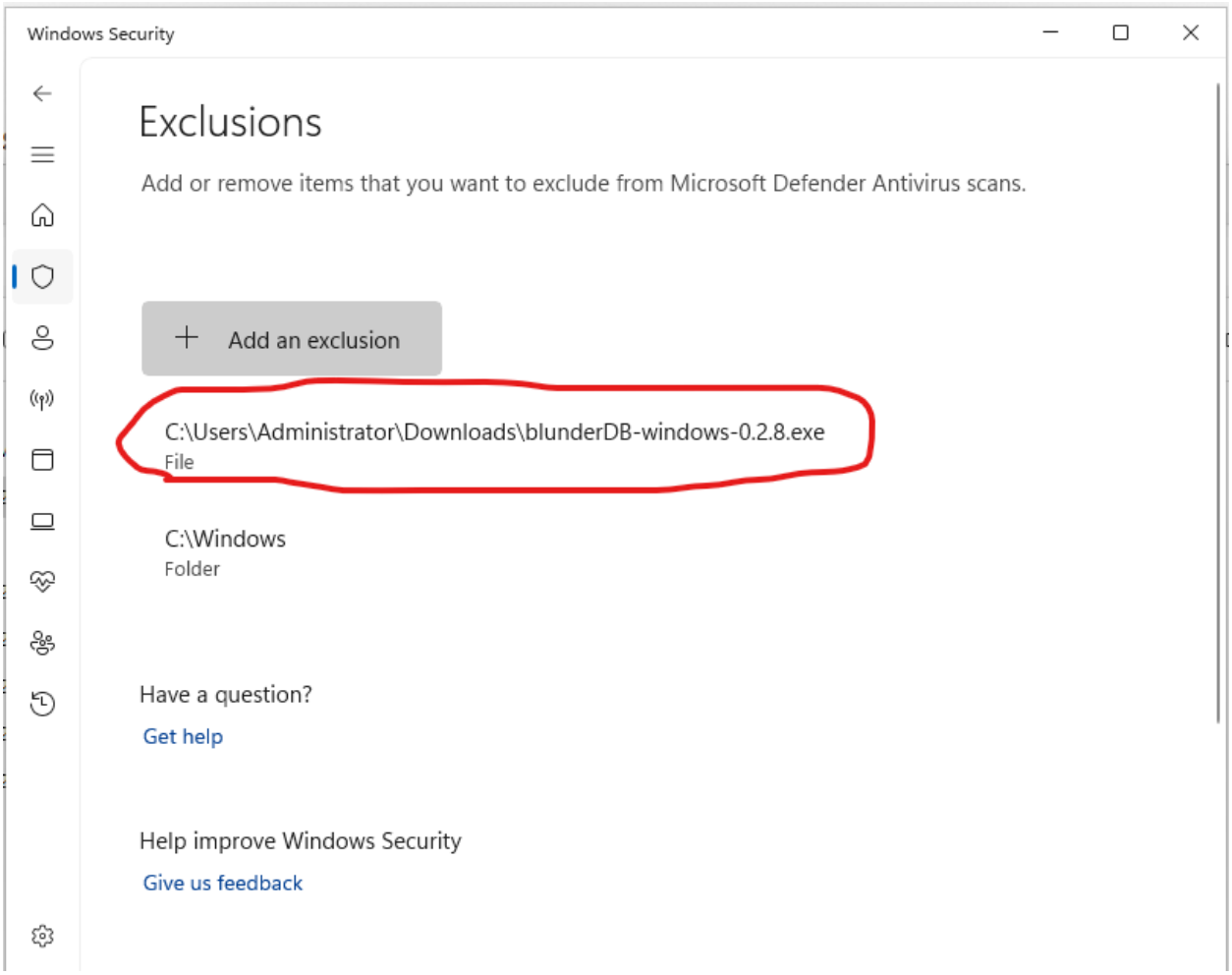

Windows Security
Security at a glance
See what's happening with the security and health of your device and take any actions needed.
Virus & threat protection
Set up OneDrive for file recovery options in case of a ransomware attack.
Set up OneDrive
Dismiss
Account protection
No action needed.
Firewall & network protection
App & browser control
The settings to block potentially unwanted apps are turned off.


Windows Security
No current threats.
Last scan: 1/22/2025 4:32 AM (quick scan)
0 threat(s) found.
Scan lasted 43 seconds
3455 files scanned.
Quick scan
Scan options
Allowed threats
Protection history
Virus & threat protection settings
No action needed.
Manage settings
Virus & threat protection updates
Security intelligence is up to date.
Last update: 1/22/2025 2:11 AM
Protection updates


Windows Security
Protect files, folders, and memory areas on your device from unauthorized changes by unfriendly applications.
Manage Controlled folder access
Exclusions
Microsoft Defender Antivirus won't scan items that have been excluded. Excluded items could contain threats that make your device vulnerable.
Add or remove exclusions
Have a question?
Get help
Help improve Windows Security
Give us feedback
Change your privacy settings
View and change privacy settings for your Windows 11 Pro device.
Privacy settings


Windows Security
Exclusions
Add or remove items that you want to exclude from Microsoft Defender Antivirus scans.
Add an exclusion
File
Folder
File type
Process
Get help
Help improve Windows Security
Give us feedback
Change your privacy settings
View and change privacy settings
for your Windows 11 Pro device.


Open
Downloads
Search Downloads
Organize
New folder
Home
Gallery
OneDrive
Desktop
Downloads
Documents
Pictures
Name
Date modified
Type
Today
xg2install
1/22/2025 2:02 AM
blunderDB-windows-0.2.8
1/22/2025 1:59 AM
File name:
All files
Open
Cancel


Windows Security
Exclusions
Add or remove items that you want to exclude from Microsoft Defender Antivirus scans.
Add an exclusion
C:\Users\Administrator\Downloads\blunderDB-windows-0.2.8.exe
File
C:\Windows
Folder
Have a question?
Get help
Help improve Windows Security
Give us feedback

Compare the displayed value with the one in the `blunderDB-windows-<version>.exe.sha256` file. The two must be identical.

On Linux or macOS:

```
sha256sum -c blunderDB-linux-<version>.sha256      # Linux
shasum -a 256 -c blunderDB-macos-<version>.zip.sha256   # macOS
```

2.11 Mac Annex: Possible Blocking of blunderDB

Note

The following concerns the macOS operating system.

Mac requires software publishing companies or independent software developers to digitally certify their applications. The developer must enroll in the Apple Developer Program by paying an annual membership fee (https://developer.apple.com/support/compare-memberships/).

Since I am sharing blunderDB for free, I do not wish to pursue these costly options. As a result, Mac will likely warn you of a potential risk or even block the execution of blunderDB entirely. The following sections explain the steps to bypass Mac's restrictions.

2.11.1 Installation of blunderDB

After downloading blunderDB, drag the downloaded file into the Applications section of your Finder. If you have already tried to run blunderDB and Mac warns you of a potential risk, follow the steps below.

2.11.2 Authorization to Run blunderDB

1. Open Finder and go to the Applications section.
2. Find blunderDB and right-click on it.
3. Select Open.
4. A warning window will appear. Click Open.
5. blunderDB will open, and you can start using it.

Note

You only need to perform this operation once. Afterward, you can open blunderDB without having to go through these steps.

2.12 Annex: Database Schema

A blunderDB database is a plain SQLite file (extension *.db*). Without blunderDB, it can therefore be opened and inspected with any SQLite editor or file browser.

2.12.1 Versioning and migrations

The database schema is **versioned**. The current schema version is **2.14.0**; it is independent of the application version and is incremented only when the internal structure changes. The schema version of an open database is shown in the **Metadata** panel (`meta` command).

Important

Always back up your *.db* file before opening a database created with an earlier version of blunderDB.

Note

Since version 0.10.0, **all schema migrations are performed automatically** when a database is opened. No manual migration command is needed. The migration is done in place: a database migrated to a recent schema can no longer be opened by older versions of blunderDB, hence the importance of backing it up first.

2.12.2 Main tables

The current schema is built around the following tables:

- **Positions and analyses**: `position` (the positions, deduplicated), `analysis` (the associated analysis data) and `comment` (the comments).
- **Matches**: `match`, `game`, `move` and `move_analysis` store the imported matches, their games, their moves and the analysis of each move.
- **Collections**: `collection` and `collection_position` (link table) group together manually chosen positions.
- **Tournaments**: `tournament` groups matches into tournaments.
- **Spaced repetition (Anki)**: `anki_deck`, `anki_card` and `anki_review_log` manage the decks, the cards and the review log (FSRS algorithm).
- **History and miscellaneous**: `command_history` and `search_history` keep the history of commands and searches, `filter_library` the filter library, and `metadata` the database metadata (including the schema version).

2.12.3 Design principles

From schema 2.0.0 onwards, several design choices speed up searches and reduce the file size:

- **Position deduplication**: each position carries a Zobrist hash and a unique index, so that the same position encountered across several imports is stored only once.
- **Denormalised filter columns**: frequently searched criteria (decision type, dice, race difference, checkers borne off, back checkers, checker-play or cube error, chances of winning, etc.) are precomputed into dedicated columns for fast filtering.
- **Bitboard prefilter**: occupancy and point-mask columns enable a very fast integer prefilter when searching for structure patterns.
- **Compact storage**: positions are encoded compactly and analysis data is compressed (zlib), which greatly reduces the file size.
- **WAL journaling**: WAL mode and tuned PRAGMAs improve read and write performance.

2.13 Annex: Statistics model — XG / gnuBG / blunderDB alignment

This page describes how blunderDB computes **PR** (Performance Rate), **Snowie Error Rate**, and **MWC loss**, and how these metrics are aligned with eXtreme Gammon (XG) and gnuBG (open reference).

2.13.1 Formal definitions

PR (Performance Rate)

PR (also called "error rate per decision" in gnuBG) measures the average error in thousandths of an equity point (millipoints, mpt) per counted decision.

$$\mathrm{PR} = \frac{\sum_i |\mathrm{error}_i|}{\mathrm{N}_{\mathrm{counted}}} \times 500$$

- The numerator is the absolute sum of EMG errors (cubeful equity) over all decisions in the scope.
- The denominator N_{counted} is the number of **counted decisions** (see below).
- The factor 500 converts equity to millipoints (1 point = 1000 mpt, but the scale is ×500 by XG/gnuBG convention — cf. `gnubg/formatgs.c:399-409`).

Snowie Error Rate

Snowie ER uses the **same numerator** as PR, but the denominator is the total number of moves of both players, forced moves included (all decisions, no filter):

$$\mathrm{SnowieER} = \frac{\sum_i |\mathrm{error}_i|}{N_{\mathrm{P1}} + N_{\mathrm{P2}}} \times 500$$

Reference: `gnubg/formatgs.c:415-424`.

Snowie ER is more stable across tools because its denominator does not depend on the forced/trivial decision filter. It serves as a cross-check metric between XG, gnuBG, and blunderDB.

Note

A player's Snowie ER is typically about half their PR, because the denominator includes moves from both players while PR only uses that player's decisions.

MWC loss (Match Winning Chance)

MWC loss expresses in percentage points of match-winning probability the cumulative effect of a player's errors. For each decision, the EMG error is converted to MWC using the MET (Match Equity Table) at the current score:

$$\text{MWCLoss} = \sum_i \text{eq2mwc}(\text{erreur}_i, \text{score}_i)$$

Reference: `gnubg/analysis.c:1449-1464`.

2.13.2 Decisions counted in the PR denominator

blunderDB follows the same exclusion rules as XG and gnuBG.

Checker plays — counted decisions

Only **unforced moves** are counted:

- A move is **forced** if the dice offer only one legal play (`cMoves == 1` in `gnubg/analysis.c:458`).
- Forced moves have zero error by definition: the player had no choice. Including them in the denominator would artificially lower PR.

Cube decisions — counted decisions

Only **close cube decisions** are counted:

- A cube decision is **close** if it lies within the equity window `[-0.16, +0.16]` around the doubling point (predicate `isCloseCubedecision` in `gnubg/eval.c:5088-5100`).
- A trivial No Double (very negative or very positive equity) is not a genuine strategic decision; including it would inflate the denominator and depress PR.

Filter summary

Decision type	Included in N_{counted} (PR)
Unforced move	Yes
Forced move	No
Close cube	Yes
Trivial No Double	No
Take / Pass	Always (these are responses to a double)

2.13.3 blunderDB ↔ XG ↔ gnuBG correspondence

Metrics are aligned within the following bounds (measured on 3 reference matches):

XG ↔ blunderDB comparison (same analysis engine)

Metric	Typical gap
Total decisions	≤ 5
Unforced moves	≤ 7
PR	≤ 0.10
MWC loss	≤ 1.0 pp
Total equity (EMG)	≤ 0.05

gnuBG ↔ blunderDB comparison (SGF import — different engines)

Metric	Typical gap	Main cause
PR (checker)	≤ 0.20	cross-engine equity differences
MWC loss	≤ 3.5 pp	incomplete close-cube data in SGF
Snowie ER	≤ 0.50	forced moves without analysis (SGF)

Note

SGF files (gnuBG) do not include alternatives for forced moves, which means blunderDB cannot detect all forced moves when importing SGF. This creates a structural gap on Snowie ER (slightly different denominator).

2.13.4 Validating your own figures

If your PR or MWC values diverge from XG figures, check the following points:

1. **Complete analyses** — PR can only be computed on positions that have an analysis. Positions without analysis count as zero error but are not included in N_{counted}.
2. **XG version** — XG may change its calculations between versions. blunderDB is aligned with the behaviour observed in recent versions.
3. **Import format** — gnuBG SGF files produce larger gaps on cube metrics (see table above) because the file does not include complete analyses for all cube decisions.
4. **Database migration** — After updating blunderDB, existing databases are migrated automatically. Make a backup before opening a database with a new version.

2.13.5 gnuBG reference

Formulas have been verified against the following source files (gnuBG repository):

- `gnubg/formatgs.c:399-409` — PR ("Error rate per decision").
- `gnubg/formatgs.c:415-424` — Snowie Error Rate.
- `gnubg/analysis.c:458-462` — Checker accumulation, exclusion of forced moves (`cMoves > 1`).
- `gnubg/analysis.c:1430-1474` — Per-decision EMG → MWC conversion.
- `gnubg/analysis.c:1449-1464` — MWC loss accumulation (`eq2mwc`).
- `gnubg/eval.c:5088-5100` — `isCloseCubedecision` predicate (0.16 threshold).

https://youtu.be/Ln7XKVFqfUk

https://youtu.be/HkY4iXjxMeI

CHAPTER

THREE

CONTACT

Author: Kévin Unger <blunderdb@proton.me>. You can also find me on Heroes under the username postmanpat.

I initially developed blunderDB for my personal use to help detect patterns in my mistakes. However, it's very rewarding to receive feedback, especially after spending a lot of time on design, coding, and debugging. So feel free to reach out to share your experiences. All (constructive) feedback is welcome.

Here are several ways to discuss:

- Join the Discord server of blunderDB: https://discord.gg/DA5PpzM9En
- Email me at blunderdb@proton.me.
- Discuss with me if we meet in a tournament.
- On GitHub.
  - Open an issue: https://github.com/kevung/blunderDB/issues
  - For bug fixes or improvement suggestions, create a pull request.

CHAPTER

FOUR

DONATE

If you appreciate blunderDB and want to support its past and future developments, you can

- buy me a drink if we have the pleasure of meeting!
- make a small donation via PayPal to the address blunderdb@proton.me

CHAPTER

FIVE

ACKNOWLEDGMENTS

I dedicate this little software to my wife Anne-Claire and our dear daughter Perrine. I would especially like to thank a few friends:

- *Tristan Remille*, for introducing me to backgammon with joy and kindness; for showing the way in understanding this wonderful game; and for continuing to support me despite my poor attempts to improve my play.
- *Nicolas Harmand*, a cheerful companion for over a decade in great adventures, and a fantastic sparring partner since he has caught the backgammon bug.